

Highlights

Offline truncated-rollout labels did not recover the online teacher on one warehouse simulator

Vittal Mukunda, Atharva Somani, Pranjal Malaiya

- On one simulator, distillation misses the teacher (J 382 vs 357).
- The distilled selector wins 0 of 12 robustness cells; the teacher wins 8.
- A two-step label is cheaper on the mean than the four-step ranker.
- 670 ms lookahead is 0.07% of a 15-minute review interval.
- Soft labels, extra samples, and 33 of 38 features are null on cost.

Offline truncated-rollout labels did not recover the online teacher on one warehouse simulator

Vittal Mukunda^{a,*}, Atharva Somani^a, Pranjal Malaiya^a

^a*Department of Industrial Engineering and Management, R. V. College of Engineering, Mysore Road, Bengaluru, 560059, India*

Abstract

Warehouse dispatching under due dates and product expiry is usually left to a priority rule. A selector can pick the rule from a queue summary by rolling every candidate forward from logged states and fitting a classifier to the cost vectors. That construction is not new. This paper asks whether the distilled classifier recovers the online truncated lookahead that produced its labels, on one warehouse simulator.

Environment, seeds and objective are held fixed on one warehouse simulator with two priced clocks. The distilled ranker (DAHS) is a gradient-boosted tree. Comparators are the online teachers, static rules, a one-step snapshot, fitted Q-iteration, and a neural PPO policy that is not a matched simulation budget. Live evaluation admits leftover arrivals at shift end (mean arrived orders 791). Production labels omitted that admit.

On 50 held-out shifts DAHS has mean composite cost $J = 382$. Online lookahead is cheaper (357–363). Always-COVERT costs 455. Fitted Q costs 398; that 4% gap is not a result about value learning. A one-step snapshot matches DAHS on the paired mean interval. A two-step label is cheaper than the deployed four-step ranker on the mean (−9.11, [−17.61, −1.35]; 21/20/9 wins-losses-ties, median 0). On a 12-cell grid the one-step teacher wins eight cells, Always-EEDD wins four, and DAHS wins none. Distillation takes 4.24 ms against 670 ms of lookahead, which is 0.07% of a 15-minute review. If a resettable simulator exists, run the teacher. If it does not, these labels cannot be generated.

Keywords: dynamic dispatching, selection hyper-heuristics, rollout, approximate policy iteration, warehouse operations, perishable inventory

1. Introduction

Order dispatching on a warehouse floor is a sequential decision problem under uncertainty: orders arrive stochastically, each carries a due date and possibly a product expiry, and a small pool of pickers must be assigned work so as to minimise late and spoiled shipments. In practice the decision is delegated to a *dispatching rule* (first-in-first-out, earliest-deadline-first, weighted shortest-processing-time, and the like) because rules are transparent, fast, and require

*Corresponding author.

Email addresses: vittalmukunda.im24@rvce.edu.in (Vittal Mukunda), atharvasomani.im24@rvce.edu.in (Atharva Somani), pranjalmalaiya.im24@rvce.edu.in (Pranjal Malaiya)

no training. No single rule dominates: the rule that minimises lateness under light load is not the rule that does so when the queue is saturated or when a burst of perishable orders arrives. A controller that *selects* the rule appropriate to the current state, a *selection hyper-heuristic* (Drake et al. 2020; Dokeroglu et al. 2024), can in principle capture the envelope of the pool without abandoning the operational advantages of rules.

The open question is how to *learn* the selector. Deep reinforcement learning (Mahmoudinazlou et al. 2025; Chen et al. 2024) is sample-hungry, and it is unstable on problems where the per-state advantage of one action over another is small relative to the return variance, which is the usual case in rule selection. Imitation of an expert dispatcher (Han and Jung 2025) needs an expert. The training signal we study has neither problem: it measures the counterfactual cost of every rule directly.

The signal is a rollout (Bertsekas 2020). Fix a state, run each candidate rule forward for a short horizon, and record the cost it incurs. Rollouts can be run online at each decision, or offline once over a corpus of states. For each state in a corpus of simulated shifts we roll out every rule, obtain a per-rule cost vector, and fit a supervised ranker to it. At deployment the ranker is a single forward pass. We retain the cost margin between rules through a soft, tempered-softmax label by default; an ablation (Section 6.8) shows the soft form is not essential.

Simulating a rule pool offline and fitting a classifier to the result is not a new training mechanism. It is related to rollout classification policy iteration and to multi-pass rule selection; Section 2 places the method inside both. The construction here is a single supervised fit to truncated-rollout labels, not an iterated policy-iteration loop, so we have not implemented RCPI.

The question this paper answers is narrower, and empirical. On one warehouse shift corpus, with environment, seeds and objective held fixed, does the distilled ranker recover the online truncated lookahead that generated its labels? Secondary comparators vary the training signal: a bootstrapped state-action value fitted from the same logged transitions (same gradient-boosted-tree class as the ranker), and a neural PPO policy that is not the same approximator class and not a matched simulation-step budget. Section 6.10 reports that comparison; Section 6.9 reports the PPO hyperparameter grid. The warehouse model carries two priced clocks, customer due date and product expiry, and Section 3.5 measures whether the second binds at a 15-minute review. The name DAHS expands to Disruption-Aware Heuristic Scheduling. The simulated warehouse is stationary: Poisson arrivals, i.i.d. processing times, no breakdowns, no cancellations. Section 6.11 perturbs parameters between labelling and evaluation; that is model misspecification, not a disruption process.

The empirical answer is no. On the default operating point DAHS beats every static dispatching rule we screened, with Always-COVERT as the static to beat. It does not beat the online teachers. On a 12-cell robustness grid five methods are frozen: the one-step teacher wins eight cells, Always-EEDD wins four, and DAHS wins none. M from 1 to 40 sits in a 0.7% band on deployed J . Under the confirmatory paired interval, $\tau = 1$ is a null against deployed $\tau = 4$; $\tau = 2$ is cheaper than $\tau = 4$ on the mean, with 21 wins, 20 losses, 9 ties and median difference 0. The deployed model remains $\tau = 4$ because that is the frozen labelling budget. Distillation is 158 times faster than the rolling-horizon teacher (4.24 ms against 670 ms). At a 15-minute review, 670 ms is 0.07% of the epoch. If a resettable simulator is

available and 670 ms is affordable, the teacher is the cheaper policy. If it is not, truncated-rollout labels of this kind cannot be generated on this corpus.

The rest of the paper is the protocol, the scoreboard, and the limitations of a single simulator.

1.1. Terminology and notation

Terms such as “corpus of simulated shifts”, “held-out shifts”, “SLA-breach rate” and “snapshot-trained ranker” are defined here, on first use in the text, or both.

Term	Meaning
decision epoch	The boundary of a review interval, where the controller acts. There are N per shift.
dispatching rule	A function that orders the waiting queue. Pickers are then assigned down that order. FIFO, EDD and the rest are dispatching rules.
selection hyper-heuristic	A controller that chooses <i>which dispatching rule to apply</i> , as a function of the current state, rather than choosing an assignment directly.
corpus of simulated shifts	A set of shifts generated from distinct random seeds, used as data. Split into three disjoint blocks: training (fits the selector), calibration (fits rule parameters such as ATC’s look-ahead scale), and test .
held-out shifts	The test block used for confirmatory tables. Ranker grid, isotonic split, ATC/COVERT k and regime K are fit on training or calibration, not on test. SHAP values and the $T_{\min}$ dwell sweep are computed on this block and are not used to select the deployed model. A <code>top5_features</code> retrain that read test-set SHAP is reported as a diagnostic in Section 6.8, not in the ablation table. The PPO sensitivity grid in Section 6.9 also read this block; the main comparison keeps the pre-declared <code>ppo_fair</code> row.
SLA	Service-level agreement — the contractual delivery commitment. An order’s SLA due time d_o is when it must ship.

Term	Meaning
SLA-breach rate	The fraction of orders shipped after d_o . Two denominators are possible and the choice matters (Section 3.3): over <i>arrived</i> orders, or over <i>completed</i> orders only. We always state which.
service-failure rate	Our primary reported KPI component: the fraction of <i>arrived</i> orders that either ship late or spoil, whether or not they were ever dispatched.
rollout	Simulating forward from a state under a fixed rule, to measure what that rule costs.
truncated rollout	A rollout stopped after τ intervals instead of running to the end of the shift.
continuation	One independently sampled future used for a rollout. The label averages over M of them.
ranker	The fitted classifier that maps an observation to a distribution over rules. A gradient-boosted decision-tree ensemble here.
snapshot-trained ranker	The ranker fitted to labels from a <i>one-interval</i> rollout, i.e. $\tau = 1$. Used as an ablation to isolate the value of looking further ahead.
soft label	The training target expressed as a distribution over rules rather than a single winner, obtained from the cost vector by a tempered softmax.
regime	A cluster of operating conditions, fitted by a Gaussian mixture over training observations. Membership probabilities are appended to the observation.
switching controller	The deployment wrapper that enforces a minimum dwell (hold a rule for $T_{\min}$ epochs) and an entropy gate (permit an early switch when the ranker is confident).
ablation	Removing or altering one component and re-measuring, to establish what that component contributes.

Term	Meaning
DAHS	Disruption-Aware Heuristic Scheduling, the selection hyper-heuristic studied here. The name is the code name. The environment in Section 3 does not model disruptions.
Notation.	
Symbol	Meaning
t, N, L, T	epoch index; epochs per shift (32); interval length (15 min); shift length (480 min)
S_t	the true state: queue with full per-order attributes, picker availability, clock
$\phi(\cdot), x_t$	the feature map, and the observation $x_t = \phi(S_t)$ the controller actually sees
u_t	the decision at epoch t — a rule from the pool
W_{t+1}	exogenous information: orders arriving in $(t, t + 1]$ and their attributes
$S^M(\cdot)$	the transition function (Section 3.4)
$\mathcal{H}, h$	the rule pool and a member of it
H_t	intervals remaining in the shift at epoch t (distinct from $\mathcal{H}$)
a_o, p_o, d_o, x_o, w_o	order o 's arrival, processing time, customer deadline, product expiry, economic weight
f_o	completion time of order o
$J; W_b, W_t, W_s, W_h$	the composite objective and its weights: breach, tardiness, spoilage, holding
τ, M, β	rollout horizon; continuations per rollout; dimensionless softmax-temperature multiplier ($T(s) = \beta \hat{\sigma}(s)$)
$T(s)$	per-state softmax temperature, in cost units
ε	per-step model error in total variation, used only as a description of the misspecification grid in Section 6.11

The remainder of the paper is organised as follows. Section 2 reviews related work. Section 3 defines the dis-

patching problem and the simulator. Section 4 presents DAHS. Section 5 describes the experimental protocol. Section 6 reports results. Section 7 is the practitioner reading; Section 8 lists limitations; Section 9 concludes.

2. Related Work

2.1. *Scope: which order-picking decision this paper addresses*

Order picking decomposes into a set of interacting planning problems — storage assignment, zoning, order batching, picker routing, and order release and dispatching (Koster et al. 2007; Gils et al. 2018). This paper addresses **order release and dispatching only**: at a fixed review epoch, which of the waiting orders are released to which picker, and in what order. Three neighbouring problems are treated as exogenous and are held fixed throughout, and because that restriction shapes every result we report, we state it explicitly rather than leaving it to be inferred.

Storage assignment and layout are fixed. They are themselves substantial design problems (Roodbergen and Vis 2006), and they enter this model only through the distribution of order processing times.

Batching is fixed at one order per pick tour. The controller never decides which orders to combine; it decides only which order a free picker starts next.

Routing and travel-time estimation are exogenous. The processing time p_o is a single realised service duration that aggregates travel, search and pick time, in the three-point form standard when only time-standard data are available (Tompkins et al. 2010).

This restriction is deliberate for three reasons. First, it matches the planning horizons of the decisions involved: layout, zoning and batching policy are typically fixed over horizons far longer than a shift, so within-shift adaptive control acts on dispatching alone (Gils et al. 2018). Second, rule selection is only a well-posed question once the downstream problems are fixed; if the action space also ranged over batch composition or route construction, the comparison between dispatching rules would confound the rule with the batching and routing policy it happens to be paired with. Third, travel-time estimation is a substantial modelling problem in its own right, and embedding an estimator inside the environment would make the rule comparison a comparison of travel models. The cost of the restriction is real — DAHS cannot exploit batching or routing synergies, and a controller that jointly optimised release and batching could dominate it — and we record this in Section 8.

2.2. *Dispatching rules and data-centric control in warehousing*

Priority dispatching rules are the standard instrument for release decisions under uncertainty: they are transparent, require no training, and execute in the time available at a decision epoch. The classical families and their properties are established (Conway et al. 1967; Pinedo 2016) — arrival-driven (FIFO), due-date-driven (EDD (Jackson 1955), minimum slack, modified due date (Baker and Bertrand 1982)), processing-time-driven (WSPT (Smith 1956)), and composite slack-and-processing indices (COVERT (Carroll 1965), ATC (Vepsäläinen and Morton 1987)). The persistent finding across that literature is that no single rule dominates: the rule minimising weighted tardiness under light load is not the rule that does so once the queue saturates or the due-date mix tightens (Pinedo 2016).

In warehousing specifically, order release and dispatching under stochastic arrivals has been surveyed by Koster et al. (2007), Boysen et al. (2019) and Boysen and Koster (2025), and the planning-problem taxonomy of Gils et al. (2018) places dispatching among the operational decisions that interact most strongly with due-date performance. The field’s recent direction is data-centric: Winkelhaus and Grosse (2020) survey the shift toward data-driven warehouse control, and learned controllers have been applied to dynamic order picking (Mahmoudinazlou et al. 2025), order batching (Cheng et al. 2024), and job-shop dispatching with an LSTM-PPO agent (Chen et al. 2024). What distinguishes the warehouse setting from the job-shop setting these methods were developed in is a second deadline clock on perishable goods. We put that clock in the objective and *measure* whether it binds at a 15-minute review (Section 3.5). It does at the epoch level; it is small at the order level (1.4% of orders, 1.3% of economic weight). The customer clock remains the dominant pivot.

2.3. *Simulation-trained selectors of dispatching rules*

Selecting a rule as a function of the shop state, with the selector trained on simulation output, is a mature line of work and predates this paper by several decades. Wu and Wysk (1988) introduced the *multi-pass* construction: at a decision point, simulate the candidate rules forward, record their outcomes, and use the result to choose. Mouelhi-Chibani and Pierreval (2010) made the selector a learned function, training a neural network on simulated states labelled by the best-performing rule so that the multi-pass simulation is paid offline and the deployed decision is a forward pass. Shiue et al. (2020) extend the same construction with reinforcement learning over the rule set. Durašević and Jakobović (2022) is a classifier over GP-evolved rules in one unrelated-parallel-machines environment, not a survey. Surveys of hyper-heuristics generally (Drake et al. 2020; Dokeroglu et al. 2024) organise this literature around the offline-learning, online-application paradigm that the present method also follows.

We correct a characterisation of this literature. Genetic programming in this literature is used predominantly to *generate* the low-level rules that are subsequently selected among, not to learn the selector (Branke et al. 2016; Nguyen et al. 2017); and prior selectors, including those in Durašević and Jakobović (2022) and Mouelhi-Chibani and Pierreval (2010), are trained essentially by supervised learning on simulation-derived labels, which is what we do as well. Shiue et al. (2020) is reinforcement learning over a rule set, not that supervised construction. The method in this paper belongs inside that tradition rather than departing from it.

2.4. *Rollout and classification-based approximate policy iteration*

A rollout policy improves a base policy by simulating each action at the current state, following the base policy thereafter, and taking the action of least simulated cost (Bertsekas 2020). Rollouts are a cornerstone of approximate dynamic programming (Powell 2022) and are typically truncated to a finite horizon for tractability (Bertsekas 2020; Goodson et al. 2017).

The construction we use (estimate action values by simulation at a sample of states, then fit a classifier to represent the improved policy) is the same family as **Rollout Classification Policy Iteration**, introduced by Lagoudakis and

Parr (2003) and developed by Fern et al. (2006), Dimitrakakis and Lagoudakis (2008) and Farahmand et al. (2015). It is not a new training paradigm. Offline rollout generation for supervised policy learning has existed for over two decades in the reinforcement-learning literature and, as Section 2.3 records, for longer than that in the scheduling literature. We also do not run the policy-iteration loop: labels are generated once under a behaviour policy and the ranker is fitted once, so the deployed controller is a one-shot imitation of truncated rollouts rather than iterated RCPI.

Two details of our instantiation differ from the standard RCPI setting, and we note them as details rather than as contributions. The classifier is fitted to the full per-action cost *vector* rather than to the arg-max alone, encoded as a tempered-softmax label distribution (Geng 2016); Section 6.8 reports an ablation showing this makes no material difference, which is consistent with RCPI practice. And the rollout is truncated at a short horizon, which is a standard truncation in the rollout tradition (Bertsekas 2020).

2.4.1. Rollout and ADP for dynamic dispatching

The operations-research literature on dynamic dispatching under stochastic arrivals is the closest methodological neighbour to this work. Klapp et al. (2018b) and Klapp et al. (2018a) formulate the dispatch-waves problem — when to release accumulated demand, given that more will arrive — a neighbouring dynamic-dispatching decision (when to release a wave), not the inner rule-selection decision studied here. Goodson et al. (2017) give a general rollout framework for finite-horizon stochastic dynamic programs, including the treatment of truncated horizons and pre- versus post-decision rollouts (Bertsekas 2020); Goodson et al. (2016) apply rollout policies to stochastic-demand routing.

Ulmer et al. (2020) set out the modelling conventions for stochastic dynamic routing and dispatching problems that are the closest operational neighbour to Section 3. Section 3 itself uses Powell’s sequential-decision notation (Powell 2019), not route-based MDPs. The same group’s work on offline–online approximate dynamic programming (Ulmer et al. 2019), on budgeting decision time under stochastic requests (Ulmer et al. 2018), and on anticipation against reactive re-optimisation (Ulmer 2019) is directly relevant to the trade-off this paper studies.

One thread deserves particular emphasis because it supersedes part of our construction. That literature does not simply truncate a rollout and discard the tail: it **approximates the remainder with a learned value function**, and in places uses learning to set the rollout horizon state-dependently. Truncating the tail and discarding it is the worse construction: a value-approximated remainder need not grow with the remaining horizon, and would likely permit a shorter τ . We do not implement it, and we record it in Section 9 as the most promising extension.

2.4.2. How this differs from value-function approximation

It is fair to ask what distinguishes this from value-function approximation or reinforcement learning, since those also learn offline from simulation. The method is **inside** the approximate-dynamic-programming family, not outside it: what is described here is one step of approximate policy iteration in which the improved policy is represented by a classifier rather than derived from a value function (Lagoudakis and Parr 2003; Fern et al. 2006).

What is learned. Value-function approximation fits $V(S)$ or $Q(S, u)$ — a scalar satisfying, approximately, a Bellman fixed point — and recovers a policy from it by one-step lookahead. Here no value function is ever formed and no fixed point is sought. The object fitted is the policy itself: a classifier over actions, trained on directly measured per-action costs at sampled states.

How error behaves. This is the difference that matters and it is why the comparison in Section 6.10 is informative. A bootstrapped value estimate propagates error through the Bellman backup, so an error at one state contaminates its predecessors and the fixed point can be reached badly or not at all. The classification construction has no backup: its error at each state is ordinary supervised generalisation error, and errors at different states do not compound. That is the mechanism our experiments are designed to isolate, holding the environment, corpus, model class, feature set and objective fixed.

What each requires. This asymmetry runs against us and we state it plainly. Value-function approximation can be fitted from logged transitions alone — the data an operating warehouse already produces. The construction here needs a simulator that can be **reset to an arbitrary state and rolled forward under a counterfactual action**, because the label is the cost of rules that were not taken. That is a strictly stronger requirement, it is the reason the method is simulator-bound, and it is why the circularity of Section 8.1 is intrinsic rather than incidental. A practitioner without a trustworthy simulator should prefer value learning from logs; the comparison in this paper is only relevant to one who has such a simulator and is deciding how to use it.

2.5. Reinforcement learning for dispatching

Deep reinforcement learning has been applied widely to dispatching and its scalability for production scheduling is under active study (Stöckermann et al. 2025; Tassel et al. 2023). Two learning-based selectors are the closest comparators and both appear as baselines here. Imitation learning of dispatching decisions (Han and Jung 2025) trains on the actions of a single expert dispatcher, and so requires an expert; the multi-pass construction instead measures the counterfactual cost of every rule and needs none. Offline reinforcement learning with maskable action-value learning (Remmerden et al. 2025) learns a value function from logged data, and is reimplemented faithfully in Section 5 as a fitted-Q baseline (Ernst et al. 2005) trained on the same logged shifts. We also include Proximal Policy Optimization (Schulman et al. 2017) under an 8000-timestep budget (Stable-Baselines3 `total_timesteps`, not review epochs), with a hyperparameter sensitivity analysis in Section 6.9. A recent review (Liu et al. 2025) frames simulation-derived self-labelling as an emerging paradigm for machine learning in scheduling.

2.6. Positioning

Given Sections 2.3 and 2.4, the mechanism at the centre of this paper is not novel, and we do not claim it. Simulating a rule pool offline and fitting a classifier to the result is RCPI in the reinforcement-learning literature and multi-pass rule selection in the scheduling literature. What we offer is an empirical study of truncated-rollout labels versus online lookahead, fitted Q and PPO on one shift corpus, not a new selector architecture.

Holding the environment, the seeds and the objective fixed, we vary how the training signal is constructed: a directly measured per-action cost vector, a bootstrapped state-action value fitted from the same logged transitions on the same gradient-boosted-tree class, and a neural PPO policy. PPO is not a matched simulation-step budget (Section 5). Section 6.10 reports the comparison; Section 6.9 reports the PPO hyperparameter grid. The warehouse instantiation carries two deadline clocks; Section 3.5 measures whether the product clock binds at this review interval. On this corpus the rollout-trained selector saturates by 50 training shifts (Section 6.3). The matched FQI budget curve in Section 6.10 is the pre-admit evaluation and is labelled as such.

The comparison is not a claim that the student recovers the teacher, that extra rollout depth buys deployed J , or that the selector is robust under configuration transfer. The default scoreboard, the horizon sweep and the robustness grid report the opposite on each of those points. We make no claim to a new training paradigm.

3. Problem Setting and Simulator

3.1. Orders, and the two deadline clocks

We consider a single warehouse shift of length T (8 hours). Orders arrive over the shift. Order o carries

symbol	meaning
a_o	arrival time
p_o	processing time — one pick tour, aggregating travel, search and pick (Section 2.1)
d_o	customer deadline: the absolute time by which the order must ship
x_o	product deadline: the absolute time after which the goods are no longer saleable. Finite for perishable orders, $x_o = \infty$ otherwise
w_o	economic weight of the order, set by its priority class

The two deadlines are distinct constraints with distinct failure modes, and they are sampled independently: missing d_o ships an order late, missing x_o destroys the goods. Either can bind first. FEFO sorts on x_o ; EDD sorts on d_o . Section 3.5 tests whether the product clock binds at this review interval.

Where the expiry of an order comes from. FEFO is conventionally a rule for issuing inventory *lots*, not customer orders, so an order-level expiry requires justification. We model the stage *after* lot allocation: an upstream allocation policy — exogenous here, in the same sense as routing and batching (Section 2.1) — has already committed specific lots to specific orders, so each order inherits a concrete expiry from the stock reserved against it. For a single-line

order that is the allocated lot’s expiry. For a multi-line order, the order ships as a unit and is therefore constrained by its most perishable component, so $x_o = \min_{\ell \in o} x_\ell$ over the allocated lines ℓ . The experiments in this paper use single-line orders, so x_o is the allocated lot’s expiry directly; the minimum-over-lines rule is the generalisation and requires no change to the controller, since the controller reads only x_o .

A fixed set of $m = 10$ pickers processes orders; a picker handles one order at a time and is busy for its processing time. Decisions are taken at the boundaries of $N = 32$ equal review intervals of $L = 15$ minutes. At each boundary the controller observes the system and chooses a *dispatching rule* from a fixed pool; that rule ranks the waiting queue, and pickers are assigned down the ranking.

3.2. The decision process, and what the controller can see

We state the problem as a **sequential decision process** in the canonical form of Powell (2019) and Powell (2022): state, decision, exogenous information, transition function, objective, before any implementation detail, and Section 3.4 then specifies the transition concretely. Readers who prefer the reinforcement-learning vocabulary may read the five elements as a finite-horizon Markov decision process; the partial-observability caveat below applies under either reading.

The sequential decision process. Decisions are taken at epochs $t = 0, 1, \dots, N - 1$, one per review interval. The **state** is

$$S_t = (Q_t, \mathbf{b}_t, t),$$

where Q_t is the multiset of waiting orders, *each carrying its full attribute tuple* $(a_o, p_o, d_o, x_o, w_o)$, and $\mathbf{b}_t \in \mathbb{R}^m$ records when each picker next becomes free. The **decision** is $u_t \in \mathcal{H}$, a rule from the pool. The **exogenous information** W_{t+1} is the set of orders arriving in $(t, t + 1]$ with their attributes. The **transition** $S_{t+1} = S^M(S_t, u_t, W_{t+1})$ is the admission–rank–assign procedure of Section 3.4. Writing $C(S_t, u_t, W_{t+1})$ for the cost accrued over the interval (Section 3.3), the problem is

$$\min_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=0}^{N-1} C(S_t, U_t^\pi(S_t), W_{t+1}) \right].$$

The observation is not the state. The controller does not see S_t . It sees

$$x_t = \phi(S_t) \in \mathbb{R}^{26},$$

a fixed-length summary listed in Appendix A. Calling x_t “the state” is wrong as terminology and, more importantly, as mechanism: ϕ records *marginal* summaries — queue length, mean and standard deviation of slack, mean processing time, counts of critical and perishable orders — and discards the *joint* distribution over per-order attributes. But it is the joint distribution that determines what a ranking rule does next, because a rule orders orders by a function of their attributes taken together.

A witness. ϕ is not injective. The constructed pair below is a searched-grid example, not a claim that every coordinate is attained under the deployed processing-time law. Consider two queues of two orders each, arriving at the same instant, differing only in which order carries the tight deadline:

$$Q^A = \{(p_{\text{short}}, s_{\text{loose}}), (p_{\text{long}}, s_{\text{tight}})\}, \quad Q^B = \{(p_{\text{short}}, s_{\text{tight}}), (p_{\text{long}}, s_{\text{loose}})\},$$

writing $s_o = d_o - t - p_o$ for slack. The two have the same queue length, the same arrival times and therefore the same ages, the same mean and standard deviation of slack, the same mean processing time, and — with s and p chosen so that the critical thresholds fall the same way — the same critical and at-risk counts. Every coordinate of ϕ agrees: $\phi(S^A) = \phi(S^B)$ exactly. The dynamics do not agree. In Q^A the binding deadline sits on the order that occupies a picker longest, so deferring it is expensive and the feasible set of on-time completions is strictly smaller than in Q^B .

Two conditions are needed for that difference to be *realised*, and both are part of the construction rather than incidental to it.

The queue must contend for a picker. A ranking expresses a preference only when something has to wait. With the deployed ten pickers and a two-order queue both orders start immediately whatever the ranking says, every rule produces the identical trajectory, and the cost gap is zero by construction. The witness is therefore built at one picker. This is not a weakening: contention is the regime in which rule selection has any effect at all, so it is the only regime in which partial observability can cost anything.

The rule must key on slack and processing time jointly. Queues Q^A and Q^B carry the *same multiset* of slacks, so a rule that ranks only on a slack-like clock will often fail to separate them. EDD sorts by due date, not slack; EEDD, MS and MDD are the closer slack-family rules. No witness is claimed for those rules. The composite rules ATC and COVERT rank on slack *and* p_o together, which is precisely the interaction ϕ discards by recording the two marginals separately. That is the sharper statement of the defect: ϕ retains the marginal distributions of slack and of processing time and destroys their coupling.

`experiments/observability_analysis.py` searches a grid of picker counts, processing times and slacks over every rule in the pool, verifies that the feature vectors coincide to machine precision before comparing anything, and reports the strongest gap. The largest ATC gap on that grid is 3.79 against -0.01 at $(p_{\text{short}}, p_{\text{long}}) = (4, 18)$ with one picker. Processing time in the simulator is `Triangular(2, 5, 12)`, so $p = 18$ is off-support. The only on-support ATC cell with a nonzero gap is $(2, 12)$, slacks $(-5, 30)$, gap 0.40 (already-late tight job). Displayed slacks $(0, 40)$ have gap 0 on-support. The pair shows that ϕ can match while ATC cost depends on the latent pairing. It is not a reachable-state regret floor, and it does not exhibit conflicting arg-mins. ϕ is still not a sufficient statistic.

Consequence: a POMDP, and a policy-function approximation. We therefore do not claim that ϕ is a sufficient statistic — it is not, and the witness settles it. The control problem is a **partially observed** Markov decision process, and the policy class we search is a policy-function approximation over the observation,

$$U^\pi(S_t) = \arg \max_{h \in \mathcal{H}} f_\theta(\phi(S_t))_h,$$

with no belief state maintained and no history beyond the lags ϕ carries explicitly. Two consequences follow, and they pull in different directions.

The unfavourable one is that partial observability *can* leave an irreducible regret floor: two states that ϕ cannot separate must receive the same action, so whenever their optimal actions differ, some regret is incurred that no amount of data or model capacity can remove. The constructed pair does not exhibit that conflict. We treat the floor as a possible consequence of ϕ , not as a result of the off-support witness. Over the training corpus we locate mutual near-neighbours in standardised ϕ -space and report how often they disagree about the cost-minimising rule, and what acting on the neighbour’s choice costs, as a share of the total benefit available from rule selection. That number is an upper bound on the part of the residual regret attributable to partial observability (3.7% in Section 8), and it is reported alongside the other limitations rather than buried.

The favourable one is more specific than it first appears. The rollout **labels** are computed from the true state S_t : the simulator holds Q_t in full, and the rollouts of Section 4.3 run the actual dynamics from it. So the supervision target is *correct*; it is only the *covariates* that are lossy. The learning problem is therefore regression with insufficient covariates — a Bayes risk induced by the feature map — and not a misspecified or biased target. This distinction matters for interpreting the results: a residual gap between DAHS and a controller with access to S_t is attributable to ϕ , not to the training signal, and enlarging ϕ (a set encoding over queued orders, for instance) is the remedy. We leave that to future work and record it in Section 9.

3.3. The objective

Let f_o denote the completion time of order o if it is dispatched. For an order still waiting at the reference horizon T we set $f_o = T + p_o$: the earliest it could possibly finish, since it still requires a full pick. With that convention every order that arrived has a well-defined outcome whether or not it was ever served, and

$$J = \sum_{o \in \mathcal{A}} w_o \left[W_b \mathbb{I}\{f_o > d_o\} + W_t \max(f_o - d_o, 0) + W_s \mathbb{I}\{o \in \mathcal{P}\} \mathbb{I}\{f_o > x_o\} \right] + W_h |Q_T|,$$

where $\mathcal{A}$ is the set of orders that arrived during the shift, $\mathcal{P} \subseteq \mathcal{A}$ are the perishable ones, and $|Q_T|$ is the queue length at shift end. Non-perishable orders have no x_o and never attract W_s . Weights are $W_b = 3.0$ (late shipment), $W_t = 0.2$ per minute of lateness, $W_s = 5.0$ (spoilage), and $W_h = 0.005$ per queued order. They are fixed before any learning and are not tuned to any method.

Three features of this objective are consequential.

Priority class now enters the objective. WSPT and ATC rank by w_o/p_o . An unweighted objective would grade those rules against a criterion the evaluation never measured (Section 6.2). Rule and objective agree.

Perishability now enters the objective. The W_s term is the only place a product deadline can be priced. Without it, “perishability-constrained” would not be a property of the optimisation problem at all.

Orders that are never served are charged. Charging an abandoned order only $W_h = 0.005$ against $W_b = 3.0$ for one served late would give a controller a factor-of-600 incentive to decline difficult orders, and a completed-orders-only breach rate would drop those orders from the metric entirely. The convention $f_o = T + p_o$ closes both gaps: an unserved order past its deadline is charged exactly as a late one, and the $+p_o$ ensures that dispatching an order onto a free picker costs precisely what abandoning it costs, with any earlier dispatch costing strictly less. Doing the work is therefore weakly optimal by construction. W_h survives strictly as a work-in-progress holding cost.

Spoilage mechanics, stated explicitly.

Does a perishable order have a distinct expiry, or is it the due date? Distinct. x_o is drawn independently of d_o (Section 3.1), so for a perishable order either clock can bind first.

What happens when an order spoils? Its goods become unsaleable at x_o . The simulator is discrete-interval: spoilage is assessed when the potential or a KPI is computed (at each review and at the shift-end reference T), not as a continuous-time event at x_o . Objective spoilage uses $f_o > x_o$, with $f_o = T + p_o$ if the order is still waiting. Dispatch after expiry does not clear W_s . The KPI predicate `is_expired_at` is stricter for unserved orders: it fires only when $T > x_o$, with no $+p_o$. The two can disagree — an unserved perishable with $p_o = 6$, $x_o = 105$ and $T = 100$ is spoiled under J and not expired under the KPI. The order is **not** removed from the queue: spoiled stock still has to be pulled and disposed of, so it continues to consume a picker when it is eventually handled. Keeping it in the queue also closes an incentive gap. If spoiled orders vanished, a controller could free picking capacity by stalling until perishables expired, which is the same class of loophole as exempting unfinished orders.

Is a spoiled order counted in the breach count? Lateness and spoilage are **separate predicates** on the same order, and an order may be neither, either, or both. When both fire, the composite cost charges both — they are distinct economic losses, a late shipment and destroyed stock. In the reported metrics they are kept apart so nothing is double-counted in a headline: the breach rate counts lateness only, the spoilage rate counts spoilage only, and the primary **service-failure rate** counts an order once if it is late *or* spoiled.

How are unfinished perishables penalised at shift end? Through the same $f_o = T + p_o$ convention as any other unserved order (above). If $T + p_o > x_o$ the order is spoiled and charged $W_s w_o$; if $T + p_o > d_o$ it is also late and charged $W_b w_o$ plus tardiness. Since shelf life is at most 120 minutes against a 480-minute shift, a perishable order still waiting at shift end is spoiled with near certainty, which is the intended behaviour: abandoning perishable stock is the most expensive thing this objective can do.

How the reported rates are calculated. Writing $\mathcal{A}$ for the set of orders admitted by the last review epoch (Section 3.4), $\mathcal{S} \subseteq \mathcal{A}$ for those dispatched, and $\mathcal{P} \subseteq \mathcal{A}$ for the perishable ones. The composite cost uses $f_o = T + p_o$ for any order still waiting at the reference horizon T . The reported *rates* do not: an unserved order is late or spoiled in the KPI only if the relevant clock has already passed at T , with no $+p_o$. That split is deliberate. Adding p_o to the KPI would count an unserved order whose due date is still in the future as a service failure.

$$\text{service-failure rate} = \frac{|\{o \in \mathcal{A} : \text{overdue or expired as of } T\}|}{|\mathcal{A}|}, \quad \text{spoilage rate} = \frac{|\{o \in \mathcal{P} : \text{expired as of } T\}|}{|\mathcal{P}|},$$

$$\text{breach rate}_{\text{arrived}} = \frac{|\{o \in \mathcal{A} : \text{overdue as of } T\}|}{|\mathcal{A}|}, \quad \text{breach rate}_{\text{served}} = \frac{|\{o \in \mathcal{S} : \text{overdue as of } T\}|}{|\mathcal{S}|}.$$

Served orders are scored at finish time; unserved orders are scored at the clock T , not at $T + p_o$. “As of T ” means that split.

The last of these is the completed-orders-only breach rate, reported here under that explicit name. Its denominator excludes every order the controller declined to dispatch, and with $W_h = 0.005$ against $W_b = 3.0$ so did an objective that charged abandoned orders only the holding term. Both rates are reported, under names that make the denominator explicit. $\mathcal{A}$ includes orders rejected at the door when the queue was at capacity; they are real demand that went unmet, and excluding them would reopen the same gap in a different place. $\mathcal{A}$ **does** include arrivals in the last open interval $(T - L, T]$: admission runs at review epochs, and a terminal admit at T enters every order with arrival time $\leq T$ into $\mathcal{A}$ as unserved. They are counted, never dispatched. Mean $|\mathcal{A}| = 791$ against a Poisson mean of $1.65 \times 480 = 792$. Those late arrivals are shared across every method.

3.4. The simulator and its parameters

The transition function S^M of Section 3.2 is realised as a discrete-interval simulation model; its construction, verification and validation follow standard practice (Law and Kelton 2000; Sargent 2013), with the input distributions fitted to a public order trace where one exists (Section 6.7) and the remaining parameters carrying explicit provenance tags below.

The environment is a periodic-review discrete-interval model. Within interval $[t, t + L)$ it (i) admits every order that has arrived **by** t , subject to a queue capacity of 200, recording overflow as *rejected demand* rather than discarding it; (ii) ranks the waiting queue with the chosen rule, evaluated at t ; (iii) assigns down the ranking to the earliest-free picker with start time $\max(\text{picker free}, t)$, stopping once no picker can start before $t + L$; and (iv) records end-of-interval statistics. Orders arriving inside the interval wait for the next epoch.

The admission rule matters. Admitting every order arriving before $t + L$ (fifteen minutes of look-ahead) and setting the start time to $\max(\text{picker free}, a_o, t)$ would reserve a picker for a not-yet-arrived order and leave it idle until that order appeared. Rules sorted by arrival never paid this cost; arrival-agnostic rules would pay it constantly (Section 6.2).

Parameters and their provenance. Every input is now either fitted to data, grounded in a cited source, or declared a design choice; none is left unexplained.

Input	Value	Provenance
Shift, review interval, pickers	8 h, 15 min, 10	Operating point
Queue capacity	200 orders	Operating point
Inter-arrival shape	Poisson (exponential inter-arrivals)	Operating point. Olist shape is lognormal; Section 6.7 and Appendix C report the fit and a frozen-ranker replay under empirical bootstrap arrivals
Arrival rate	1.65 orders/min nominal	Operating point; swept in Section 6.5
Processing time p_o	Triangular(2, 5, 12) min	Literature: three-point time standard for manual picker-to-parts picking (Tompkins et al. 2010; Koster et al. 2007)
Customer window $d_o - a_o$	Triangular(15, 45, 90) min	Operating point. AIC on the Olist purchase-to-estimated-delivery sample selects lognormal, not triangular (Appendix C). The campaign uses this triangular envelope at warehouse scale; it is not the MLE
Shelf life $x_o - a_o$	Triangular(20, 60, 120) min	Design parameter; no public trace carries expiry. Swept in Section 6.11
Perishable fraction	0.20	Design parameter; varied by scenario
Priority classes and weights	{low, medium, high} at (0.50, 0.35, 0.15), $w_o \in \{1, 2, 4\}$	Design parameter

Fitting is the right operation on a trace that exists, and Section 6.7 reports the fits, the candidate families compared by AIC, and, for processing time, where the trace carries no warehouse pick-time field, the reason no fit is attempted.

3.5. Does perishability bind at a 15-minute horizon?

A model can carry a product deadline without that deadline ever changing a decision. Since we claim the setting is perishability-constrained, we test the claim directly rather than assert it. The criterion is decision-relevance: in what

fraction of decisions does delaying an order by one review interval alter its outcome?

At each epoch t , a waiting order has exactly two options available to the dispatcher — served now, completing at $t + p_o$, or deferred, completing no earlier than $t + L + p_o$. Call the order **expiry-pivotal** at t when those two straddle its product deadline,

$$t + p_o \leq x_o < t + L + p_o,$$

so that one interval of delay is the difference between saleable goods and waste, and **due-pivotal** when the same holds for d_o .

Three conditions were fixed *before* the diagnostic was run, and all three had to hold for the framing to stand: at least 5% of decisions carrying an expiry-pivotal order; at least 10% of perishables having their product clock bind before their customer clock; and the choice of rule moving the spoilage count at at least 10% of epochs. Had any failed, the correct response was to drop the perishability framing and the expiry-aware rules, and to report that instead.

Table 1. Perishability decision-relevance, 30 calibration shifts $\times$ 8 behaviour rules (the nine-candidate SCREENING_POOL without FIFO: EDD, EEDD, FEFO, WSPT, ATC, MS, MDD, COVERT). 7,440 recorded decision epochs (empty queues omitted). These are repeated measures on the same 30 shifts, not 7,440 independent shifts. The shipped deployed pool `resolve_pool` is six rules; reproducing this table from `current config.yaml` will not recover it.

Quantity	Measured	Threshold
Decisions with an expiry-pivotal order in queue	35.5%	$\geq 5\%$
Perishables whose expiry binds before their due date	27.6%	$\geq 10\%$
Epochs where the rule choice changes the spoilage count	91.0%	$\geq 10\%$
Queued perishables that are expiry-pivotal	7.3%	—
All orders that are expiry-pivotal	1.4%	—
Share of economic weight on expiry-pivotal orders	1.3%	—
Mean spoilage <i>weight</i> spread across rules, when discriminating	3.97	—

All three conditions are met at perishable fraction 0.20, so the constraint is real at this horizon and the framing

stands at the operating point we ran. We did not rerun Table 1 at 0.05 or at 0.01. The Olist food/drink share is 0.0099, about one twentieth of 0.20. If the 35.5% expiry-pivotal rate scaled linearly with the perishable fraction, it would sit near 1.8% at 0.01 and would miss the 5% mark. That scaling is untested. Section 6.11 sweeps shelf life by $\times\{0.8, 1.0, 1.25\}$ around this inflated fraction; it is not a perishable-fraction sweep.

Two qualifications belong with that conclusion rather than after it. The *marginal* rate is small — only 1.4% of individual orders are expiry-pivotal at any given epoch, carrying 1.3% of economic weight — so perishability is not the dominant cost driver; the customer clock is, with 95.1% of epochs carrying a due-pivotal order against 35.5% carrying an expiry-pivotal one. The 3.97 figure is `mean_spoilage_spread_when_discriminating` from `pivotality_summary.json`: a mean difference of economic *weight* ($w_o \in \{1, 2, 4\}$), not an order count. The diagnostic stores the weight spread and discards the count. What makes the clock decision-relevant is concentration and frequency: the pivotal orders cluster, so a third of all decisions have at least one in the queue, and at 91.0% of epochs the choice of rule moves realised spoilage weight. A constraint that changes the outcome at nine epochs in ten is binding on the controller whatever share of orders it touches.

3.6. The rule pool

The pool is not a convenience sample. Candidates span the four information sources a dispatcher can key on, so that “no single rule dominates” is a structural property of the design space rather than an empirical accident. Nine candidates are screened (Section 6.1); the six that survive are marked.

Rule	Keys on	Source	Retained
FIFO	arrival only	zero-information control	—
EDD	customer deadline	Jackson (1955)	✓
EEDD	both deadlines: $\min(d_o, x_o)$	this paper (see below)	✓
MS	customer deadline slack	Conway et al. (1967)	✓
MDD	customer deadline, degrading to SPT when past due	Baker and Bertrand (1982)	✓
FEFO	product deadline only	classical lot-issuing rule	—
WSPT	weight and processing time	Smith (1956)	—
ATC	slack $\times$ processing, exponential discount	Vepsäläinen and Morton (1987)	✓
COVERT	slack $\times$ processing, linear truncation	Carroll (1965)	✓

Four points about the pool.

FEFO is not a due-date rule, and neither rule alone is right. FEFO sorts on x_o ; EDD sorts on d_o . That is EDD’s clock, not FEFO’s. With two deadlines, *neither* single-clock rule is the sensible one. EDD ignores expiry entirely. FEFO ranks on x_o , which is ∞ for the 80% of orders that are not perishable, so it dumps every non-perishable order behind every perishable one — on this order mix that is close to a strawman, and it is why the FEFO mask (Section 4.3) had to exist at all.

The rule a scheduler would actually write, told that orders carry two deadlines, sorts ascending on the **effective deadline** $\min(d_o, x_o)$ — whichever clock binds first. We call it **EEDD**, and it is in the pool because leaving it out would have made “expiry-awareness buys nothing” an artefact of testing only a degenerate expiry rule. Section 6.1 reports what it earns.

ATC is calibrated, not assumed. WSPT is exactly the $k \rightarrow \infty$ limit of ATC: as the look-ahead scale grows, $\exp(-\text{slack}/(k\bar{p})) \rightarrow 1$, leaving the WSPT index w_o/p_o . A correctly calibrated ATC therefore cannot be beaten by WSPT, and an unfitted k fixed at 2.0 would invert that ordering. We calibrate k twice, on a calibration corpus disjoint from both training and test shifts: once for **standalone** use, because ATC is itself a reported benchmark and benchmarking an uncalibrated rule understates it; and once for **portfolio contribution**, which is the quantity that matters when the rule sits inside a selector. COVERT’s scale is calibrated the same way. Section 6.1 reports both values and the deployed one.

Screening is by marginal contribution, not win rate. A rule earns its slot by covering states the others handle badly. We therefore report, per rule, both its win rate and the increase in achievable cost if it were removed from the pool, with a bootstrap interval on the latter. A rule with a high win rate and zero marginal contribution is redundant; a rule with a low win rate and positive marginal contribution is a specialist worth keeping. Win rate alone cannot distinguish the two. This is also how we answer what **FIFO** contributes in a due-date-driven setting: it enters as the zero-information control and the screen reports whether it earns its place.

4. The DAHS Method

4.1. Overview

DAHS is an offline-learned, online-applied selection hyper-heuristic. Training proceeds in four stages: (1) generate a corpus of simulated shifts under a state-covering behaviour policy; (2) for every decision state, run a truncated-horizon rollout of each rule and record the per-rule cost vector, from which a training label is formed; (3) discover a small set of operating *regimes* and append regime-membership features; (4) fit a calibrated gradient-boosted ranker to the rollout-derived labels. At deployment a lightweight *switching controller* wraps the ranker with a dwell constraint and an entropy gate. The expiry-rule mask is a no-op on the screened pool (Section 4.3). We describe each stage in turn.

4.2. The observation vector

At each decision epoch the controller receives $\phi(S_t)$, a 26-dimensional summary of the true state (Section 3.2) covering queue, resource, customer- deadline, product-deadline, arrival, history and temporal context. It is an observation, not a state, and Section 3.2 gives the explicit pair of queues it cannot separate. Appendix A lists every feature with its group and the source or design rationale.

Two features that would make the matrix singular are omitted. `time_to_next_expected_carrier` is $1/\lambda$ and therefore **constant** within a configuration; `intervals_remaining` is an exact affine function of `interval_index_in_shift`, the two summing to N by construction. Together they make the feature matrix exactly singular, which silently corrupts the regime layer’s model selection (Section 4.4). Three expiry-pressure features are included, since the product deadline enters the objective and a selector cannot act on a constraint it cannot see. Appendix A also reports the correlation and variance-inflation analysis, and Section 6.8 records a test-set SHAP diagnostic that is not treated as a held-out ablation.

To $\phi(S_t)$ DAHS appends the regime-membership posteriors of Section 4.4.

4.3. Rollout-informed training labels

The supervisory signal is generated as follows. We simulate a corpus of training shifts under a state-covering behaviour policy, giving one decision state per review epoch per shift. For each state s_t we form a label over the pool by **multi-sample truncated rollout**:

1. Walk the shift forward to epoch t , so that s_t is the true state S_t with its full queue.
2. Draw M independent continuations. Continuation m freezes the realised history at t and resamples the *unrealised* future — arrivals after t and their attributes — from a stream seeded by `SeedSequence([base_seed, shift_seed, t, m])`, not on the rule under test.
3. For each rule h , hold h fixed for the next τ epochs of continuation m (open-loop truncated simulation of that rule — not one improving action followed by a different base policy), and record the cost $\hat{J}_{h,m}^\tau(s_t)$ accrued over that window. Average:

$$\hat{J}_h^\tau(s_t) = \frac{1}{M} \sum_{m=1}^M \hat{J}_{h,m}^\tau(s_t), \quad \widehat{\text{se}}_h(s_t) = \frac{\hat{\sigma}_h(s_t)}{\sqrt{M}}.$$

4. Convert the cost vector into a probability distribution by a **tempered softmax** at a state-dependent temperature:

$$p_h^\tau(s_t) = \frac{\exp(-\hat{J}_h^\tau(s_t)/T(s_t))}{\sum_{h'} \exp(-\hat{J}_{h'}^\tau(s_t)/T(s_t))}, \quad T(s_t) = \beta \hat{\sigma}(s_t),$$

where $\hat{\sigma}(s_t)$ is the standard deviation of that state’s own cost vector across the pool and β is a single dimensionless multiplier fitted once on the training corpus.

Why $M > 1$. If every stochastic quantity is pre-sampled when a shift is generated, every rule sees the identical realised future belonging to the shift seed. The label then records which rule was best **in hindsight on one path**,

not which had the lowest expected cost, and the rollout variance is identically zero. Hindsight-optimal on one path and lowest-in-expectation are different quantities. The estimator below records a per-cell standard error $\widehat{\text{se}}_h$ alongside every label so the residual noise is reported rather than assumed away.

Terminal admit and the labels. Live evaluation admits leftover arrivals at T (`sim.terminal_admit: true`). Production labels were generated with that flag off and were not regenerated for the completeness run. Truncated windows that exhaust the remaining shift *do* call the terminal admit when the flag is on, so the guarantee that truncated rollouts are independent of the flag is false for last- τ epochs. The default comparison in Section 6.2 is an eval-only refresh of frozen controllers, not a relabelled campaign.

Common random numbers. The continuation seed depends on the shift, the epoch and the sample index, and deliberately **not** on the rule under test. Every candidate is therefore scored against an identical set of M futures. The comparison is paired, and since the tempered softmax reads only *differences* between rules, the relevant variance is that of $\hat{J}_h^\tau - \hat{J}_{h'}^\tau$, which is far smaller than that of either term: the shared arrival shock that dominates a single rollout’s cost cancels. This is why a modest M suffices where independent sampling would need an order of magnitude more. Section 6.4 sweeps M and reports where the label stabilises.

Cost. Walking each shift forward once and branching at each epoch costs $O(N \cdot |\mathcal{H}| \cdot M \cdot \tau)$ interval-steps per shift. A labeller that replayed each shift from $t = 0$ for every epoch and every rule would be quadratic in N . The linear walk is what makes $M > 1$ affordable. Section 6.12 reports the measured totals.

Why the temperature is per state. A single temperature fitted across the whole corpus is dominated by expensive late states. Charging orders that are never served (Section 3.3) makes a state’s cost scale with its outstanding work, so the spread of the per-rule cost vector grows through a shift — by two orders of magnitude between the first epochs and the last, measured on the corpus. A single temperature is then set by the expensive late states, and the early ones, where the rules genuinely differ by little, come out close to uniform; the median label entropy sits above any target band at every multiplier. Dividing by the state’s own spread asks the question the ranker actually has to answer — *how much better is the best rule than the others, here* — and leaves the absolute cost of the state, which the ranker does not predict, out of the label. States where every rule ties carry no information at any temperature and fall back to the uniform.

β is then selected once by a one-dimensional bisection on the training corpus, so that the median training-label entropy falls in a target band expressed as a fraction of $\log |\mathcal{H}|$ rather than in absolute nats — sharp enough to be informative, soft enough to retain the cost margin. On the regenerated corpus ($|\mathcal{H}| = 6$, $M = 20$, $\tau = 4$, 250 train shifts) the search selected $\beta = 0.470$ under per-row temperature and achieved median train-row entropy 0.638 nats against the band $[0.387, 0.905]$. The reported value is not a node of `beta_grid`. The same β is applied to the test and calibration corpora, which would otherwise not be on the ranker’s scale.

Two corrections are applied consistently in both labelling and deployment: when the perishable fraction is below 0.05 the FEFO mass is zeroed and the distribution renormalised, because FEFO cannot rank a queue with no product deadlines; and, *for the test corpus only*, states whose maximum label probability falls below $\theta = 2.2/|\mathcal{H}|$ are filtered

out as genuinely ambiguous decisions. The threshold scales with the pool for the same reason the entropy band does.

The mask is inert in the deployed model, and we leave it in place rather than delete it. Screening dropped FEFO (Section 6.1), so there is no expiry-only rule left for the mask to suppress and it is a no-op on the deployed pool. It is retained because it is a property of the *labelling and deployment contract*, not of one pool: any pool containing a rule that ranks on x_o alone needs it, and removing the code would silently reintroduce the defect if FEFO ever returned. It should not, however, be counted as a working component of the method as deployed. On the screened pool it is a no-op. EEDD, which ranks on $\min(d_o, x_o)$, needs no mask: on a queue with no perishables it degrades continuously to EDD rather than becoming undefined.

Where the corpora come from. Shift seeds are drawn once from a single `SeedSequence` and partitioned into three contiguous, disjoint blocks: training, calibration and test. The calibration block exists so that rule hyperparameters (ATC’s and COVERT’s look-ahead scales, Section 3.6) can be fitted without touching either of the other two. Each shift contributes one decision state per review interval, so a block of n shifts yields $32n$ states: the test corpus of 50 shifts gives $50 \times 32 = 1600$ states before filtering. On the labelled test corpus 1525 of 1600 test states survive ($\theta = 2.2/|\mathcal{H}| = 0.367$); 33.4% of training epochs have a best/second-best gap below one pooled standard error at $M = 20$. The filter is applied to the test corpus only, and never to the training corpus, so no training state is discarded for being difficult.

The horizon is fixed at $\tau = 4$ for the deployed model; Section 6.4 studies the choice.

By default DAHS uses the soft label above: a state where EEDD and ATC are near-tied produces a near-uniform target over those two rules, and the ranker is trained to reproduce that uncertainty rather than to guess. The arg-max of the cost vector — a *hard* label — is the natural alternative, and Section 6.8 reports an ablation that finds the two equivalent. We describe the deployed (soft) model here and treat the label’s form as a design choice rather than as a contribution.

The deployed model truncates the rollout at $\tau = 4$ of up to 32 intervals. We do not claim a bound on the truncation remainder: arrivals are Poisson, so a finite per-interval cost scale is not proved, and the implemented window cost is a terminal potential that can be negative. Horizon is treated as an empirical knob (Section 6.4). Model error is treated by labelling under nominal dynamics and evaluating frozen controllers under perturbation (Section 6.11).

4.4. Regime discovery

Warehouse shifts pass through qualitatively distinct operating regimes — a quiet opening, a saturated mid-shift, a perishable burst. DAHS makes this explicit. A Gaussian mixture model is fit to the training-state features; the number of components is chosen by BIC over $K \in \{2, 3, 4, 5, 6, 7, 8, 10, 12\}$ with five EM restarts per K . On the regenerated corpus BIC selects $K^* = 12$, the **upper endpoint of the grid** (mean pairwise ARI 0.970, above the 0.85 stability threshold). Because the selected K sits on the boundary, the grid — not a turning point in the data — chose the number of regimes; we report that rather than treating twelve as a discovered structure. The twelve soft regime-membership posteriors are appended to $\phi(S_t) \in \mathbb{R}^{26}$, so the ranker sees a 38-dimensional vector. Regime discovery

remains a lightweight component; Section 6.8 ablates it.

4.5. The calibrated ranker

The ranker is a gradient-boosted decision-tree classifier (Chen and Guestrin 2016) with an $|\mathcal{H}|$ -class soft-probability output ($|\mathcal{H}| = 6$ after screening). The soft target is fitted by an inverse-entropy-weighted replication of each training state across the six classes, which makes the training objective the Kullback–Leibler divergence between the predicted distribution and the soft label and down-weights states whose labels are near-uniform (and therefore carry little discriminative signal). The hard-label variant of Section 6.8 instead uses a standard one-hot cross-entropy; the rest of the pipeline — feature set, cross-validation, calibration — is identical. Hyperparameters are selected by 5-fold cross-validation with folds grouped by shift, so no shift contributes states to both a training and a validation fold. The reference point for the cross-validated soft cross-entropy is the uniform label, $\log |\mathcal{H}|$, which moves with the screened pool. The selected configuration is `max_depth = 6`, `n_estimators = 200`, `learning_rate = 0.03`, with mean grouped-CV soft cross-entropy 0.875 against the uniform baseline $\log 6 \approx 1.79$.

Tree ensembles are not probability-calibrated out of the box. DAHS post-processes the ranker output with isotonic regression fit on a held-out 20% of training shifts. The design target on expected calibration error is 0.05; Section 6.6 reports the achieved value. Missing the bar does not change the deployed model.

4.6. The switching controller

At deployment the calibrated ranker emits, each interval, a distribution over the retained pool $\mathcal{H}$. A thin *switching controller* maps that distribution to an action. It (i) applies the same expiry-rule mask used at labelling time, which is a no-op on the screened pool (Section 4.3); (ii) enforces a minimum dwell of $T_{\min} = 2$ intervals, counting the switch epoch (`_dwell_remaining = t_min - 1`), to prevent operationally disruptive rule thrashing; and (iii) overrides the dwell and switches immediately when ranker entropy is below half the maximum ($H < 0.5 \log |\mathcal{H}|$, ≈ 0.90 nats at $K = 6$). We deliberately frame the controller as a *stability and constraint-enforcement guardrail*, not a performance driver — and Section 6.8 reports, honestly, that removing it slightly *improves* the headline KPIs. Its role is to make the policy deployable (bounded switching), not to win the comparison; the ablation quantifies the small KPI price of that guardrail.

5. Experimental Setup

Test shifts. All methods are evaluated on the same 50 held-out shift seeds, disjoint from the 250 training shifts and fixed once. Every reported KPI is a mean over these 50 shifts.

Scenarios. Beyond the default operating point, three scenarios stress the method: *low load* (arrival rate 1.0, no perishables), *balanced* (rate 1.5), and *high-load-perishable* (rate 2.2, perishable probability 0.4, customer windows

scaled to (12, 36, 72) minutes — $0.8\times$ the default triangular). Scenario parameters were fixed before evaluation and are not tuned per method.

Baselines. We compare DAHS against the static rules retained by the screen of Section 3.6; **snapshot_xgb**, an ablation identical to DAHS but with the rollout horizon collapsed to $\tau = 1$, isolating the value of the horizon; **LinUCB** (Li et al. 2010), a contextual bandit, with features standardised (LinUCB updates on the test shifts and is therefore not a frozen peer); **PPO** (Schulman et al. 2017) at 8000 training timesteps (`total_timesteps`; not review epochs), with the hyperparameter sensitivity analysis of Section 6.9 (not a matched simulation budget: labelling uses millions of interval-steps); and **offline_fqi**, a faithful offline reinforcement-learning competitor — fitted Q-iteration (Ernst et al. 2005) with FEFO action masking — a documented no-op when FEFO is absent from the pool — an instance of the maskable-action-value family of Offline-LD (Remmerden et al. 2025). It trains on the same logged shifts as DAHS, under the same behaviour policy and per-interval reward, and uses the same gradient-boosted-tree model class and the same feature set as the DAHS ranker, so that the comparison isolates the training signal — a directly measured per-rule cost vector against a single bootstrapped value — from the function approximator and from the state representation. Section 6.10 analyses it, together with the action-coverage diagnostics that determine whether the comparison is clean.

The teacher: rolling-horizon rollout MPC. DAHS is trained to reproduce a τ -step rollout, so the controller that simply *runs* that rollout online is the natural reference. **rolling_mpc** evaluates every rule over τ intervals, averaged over independent continuations that use the same scoring function as Section 4.3 (not the labelling RNG; teacher evaluation uses a separate seed offset), commits the arg-min rule for one interval, discards the remainder of the plan, and replans. Labelling uses $M = 20$ continuations; the online teachers use $M = 5$ so that a 50-shift evaluation is affordable. The scoring rule is the same potential difference, but the Monte Carlo budget is not, so a gap between **rolling_mpc** and DAHS mixes function-approximation error with estimator noise and cannot be read as a pure distillation gap. Raising the online teachers to the labelling budget $M = 20$ does not close it: **rolling_mpc** moves from 363.42 to 363.76 and **greedy_mpc** is unchanged at 356.98. At non-terminal epochs a $\tau = 1$ step never admits $(t, t + L]$, so **greedy_mpc** is invariant to M . At the last epoch, with `terminal_admit` on, `run_with_policy` then admits the resampled tail; the reported M -invariance of **greedy_mpc** is empirical. **greedy_mpc** beating **rolling_mpc** on default J is consistent with a noisy $M = 5$ arg-min at $\tau = 4$. The one-step controller is retained as its $\tau = 1$ special case.

This baseline is what makes the paper’s central claim falsifiable. *How much does distillation lose?* The gap between **rolling_mpc** and DAHS at the same τ is the price of replacing a lookahead with a forward pass. *What does it buy?* We report per-decision latency for both, so the amortisation appears as a measured ratio rather than an argument. *Can the student beat the teacher?* Section 6.2 reports that it does not: both online lookaheads are cheaper, with paired intervals excluding zero. A fitted selector could in principle regularise noisy rollouts; that is not what happens here. *And is the horizon the mechanism?* Comparing **rolling_mpc** at $\tau = 1$ against $\tau = 4$ separates the value of lookahead depth from the value of learning.

Objective and metrics. Every learned method optimises the composite cost J of Section 3.3, and J is the primary

metric of comparison. The breach count is only one of the four terms in the objective. Making J primary removes a reporting incentive: a method could improve one component at the expense of another that went unreported.

That every learned method is scored against the same J is the evaluation contract, not a claim that every arm sees the same last-interval increment. The objective is defined once, in a single module. Rollout labels are truncated-horizon estimates of that J — truncation is the method, not a defect to be closed. `sim.terminal_admit` gates DAHS `run_with_policy` and the static/teacher eval loop. PPO and fitted Q-iteration always fold leftover arrivals into the last-step potential; they do not read the flag. LinUCB’s payoff is `env.potential()` after the *next* epoch’s `observe()`, so it includes newly admitted arrivals, and `reset()` between shifts drops the last interval. Frozen DAHS labels were generated with the flag off and omit last-window leftovers; live $|A| \approx 791$ tables align with FQI/PPO on that interval, not with those labels. The static rules optimise nothing and are evaluated against J . The objective is written once, in a single module shared by the labeller and the evaluation harness.

Alongside J we report its decomposition, so no component can hide: the **service-failure rate** — the share of *arrived* orders that ship late or spoil, whether or not they were ever dispatched — together with mean tardiness, spoilage rate, throughput, unserved and rejected demand, and picker utilisation. We also report the breach rate over completed orders only, under that explicit name. Uncertainty is quantified by 10,000-resample bootstrap 95% confidence intervals. **Confirmatory test for paired composite cost.** A warehouse pays total cost, not median cost, so the confirmatory functional is the mean. A method is declared different from DAHS on J when the paired 95% bootstrap interval of the mean difference excludes zero. Wilcoxon signed-rank p -values with Benjamini–Hochberg control are a sensitivity analysis on the sign and median of the paired differences and are not the confirmatory rule. They are reported the same way for every claim, not chosen per claim. Several paired differences on this corpus are tail-driven (Always-EEDD: 21 wins, 7 losses, 22 exact ties; $\tau = 2$: median difference 0). A mean interval can therefore exclude zero when the median does not, and the reverse. When that happens we still follow the mean interval, and we label the claim as a statement about mean cost. Service-failure rate uses the same interval rule when it is the claim being tested.

6. Results

6.1. Rule calibration, screening, and complementarity

If one rule were best everywhere, selection would be pointless. Establishing that it is not requires calibrated rules, a screen that distinguishes redundant rules from specialists, and evidence of complementarity across the **state space** rather than across instances.

This section is complete: it runs on the 30-shift calibration block, which is disjoint from both training and test, and it does not depend on the trained selector. Its results therefore stand ahead of the rest of Section 6.

6.1.1. Calibration

Table 2. Look-ahead scale calibration, 30 calibration shifts, $M = 5$ continuations under common random numbers. Standalone cost is the composite objective with the rule used alone; portfolio marginal is the cost increase when the rule is removed from the pool at that k .

Rule	$k_{\text{standalone}}^*$	cost at k^*	$k_{\text{portfolio}}^*$	marginal at k^*	deployed
ATC	1.5	459.2	3.0	0.92	3.0
COVERT	4.0	404.3	4.0	3.21	4.0

The portfolio value is deployed, because the pool is what ships; both are reported because ATC is also a standalone benchmark. The Always-ATC row of the default comparison (Section 6.2) uses the portfolio scale $k = 3.0$, not the standalone $k^* = 1.5$. A standalone Always-ATC at $k = 1.5$ has now been evaluated on the same 50 test shifts and costs $J = 526.99$, against 560.76 for the portfolio scale $k = 3.0$: refitting k recovers 33.8 of composite cost, but Always-ATC at its own optimum remains far above DAHS (382.27).

This settles the WSPT/ATC inversion. ATC’s standalone cost is U-shaped in k with a minimum of 459.2 at $k^* = 1.5$, and rises monotonically thereafter to 1004.2 at $k = 20$ — a factor of 2.19. Since WSPT is exactly the $k \rightarrow \infty$ limit (Section 3.6), that curve *is* the ATC-to-WSPT interpolation, and it says a fitted ATC beats WSPT by more than two-fold on this problem. An unfitted $k = 2.0$ inverts that ordering. The two fitted values also differ by a factor of two ($k_{\text{standalone}}^* = 1.5$ against $k_{\text{portfolio}}^* = 3.0$), which is the concrete case for calibrating both: the scale that makes ATC best *alone* is not the scale that makes it most useful *inside a pool*, where its job is to cover states the other rules handle badly.

6.1.2. Screening

Table 3. Pool screening on the same corpus. Screening is by marginal contribution to composite cost. Marginal contribution is the increase in achievable cost when the rule is removed, with a percentile bootstrap interval; a rule is retained when that interval excludes zero.

Rule	win rate	marginal contribution	95% CI	retained
EEDD	0.650	5.403	[4.840, 5.991]	yes
COVERT	0.145	2.047	[1.613, 2.527]	yes
MS	0.070	0.248	[0.119, 0.412]	yes
ATC	0.055	0.086	[0.032, 0.155]	yes
MDD	0.011	0.039	[0.014, 0.070]	yes
EDD	0.068	0.007	[8.7×10^{-7} , 0.021]	yes
FIFO	0.001	0.000	[0.000, 0.000]	no

Rule	win rate	marginal contribution	95% CI	retained
WSPT	0.000	0.000	[0.000, 0.000]	no
FEFO	0.000	0.000	[0.000, 0.000]	no

FIFO earns nothing. It is the cost-minimising rule at 0.1% of decisions and its marginal contribution is identically zero. As the zero-information control in a due-date-driven setting that is the expected result, and it is the direct answer to the question of what FIFO was doing in the pool: nothing, and it is dropped. Cause 2 in Section 6.2 is why an arrival-sorted rule can look cheap under non-causal admission: every arrival-agnostic rule is penalised and FIFO is not.

WSPT earns nothing either, and that is consistent rather than surprising. A calibrated ATC dominates it by construction, so once ATC is fitted, WSPT is a strictly worse member of the same family and contributes nothing at the margin.

FEFO’s failure is not evidence against expiry-awareness. FEFO ranks on x_o , which is infinite for the 80% of orders that are not perishable, so it sorts every non-perishable order behind every perishable one. It contributes nothing because it is a bad rule on this order mix, not because the product clock is uninformative — and reading its zero as “expiry-awareness buys nothing” would have been the wrong inference. EEDD, which reads $\min(d_o, x_o)$, settles it: it wins 65% of decisions with a marginal contribution of 5.403, two and a half times COVERT’s and by far the largest in the pool. Expiry-awareness matters a great deal on this problem; the screen was rejecting a poor implementation of it.

EEDD wins 65% of decisions. That is above a 0.60 concentration mark we wrote down before looking at the screen. We did not drop the rule to get under the mark, and we do not treat the mark as a gate that changed the rest of the campaign: the question this paper asks is what distillation is worth on the screened pool, and EEDD is the pool’s dominant member. A pool this concentrated leaves a selector little room. Whether that room is enough on J is what Section 6.2 measures, and the per-cell oracle gap below bounds it.

6.1.3. Complementarity

Win rate over a grid of the two state dimensions that govern the decision (queue length and deadline pressure, mean slack), in quantile bins, is the test of complementarity. A pool is complementary when different rules own different cells. Figure 1 reports that grid.

Table 4. Cell ownership and the oracle gap over the 4x4 grid (960 decision epochs from the calibration corpus).

Quantity	Value
Grid cells	16
Cells owned by EEDD	15

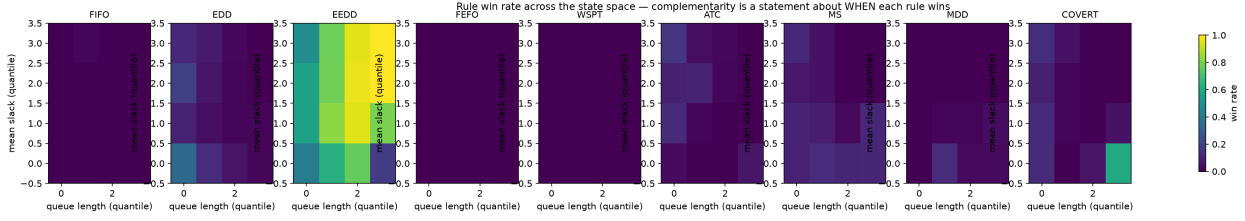


Figure 1: Rule complementarity over the state space: win rate of each retained rule across a grid of queue length (quantile bins) against deadline pressure (mean slack, quantile bins). Complementarity means different rules own different cells.

Quantity	Value
Cells owned by COVERT	1
Decisions won by the best single rule (always EEDD)	65.00%
Decisions won by the per-cell oracle	72.29%
Oracle gap over the best single rule	7.29 pp

We report this against ourselves. One rule owns fifteen of sixteen cells. At this resolution the pool is not complementary, and a selector that reads only queue length and deadline pressure could beat “always EEDD” on at most 7.29 percentage points of win rate. That is a small opening, and it is the correct place to say so, before Section 6.2 rather than after it.

Two qualifications matter for reading it, and neither rescues the number so much as bound it in the other direction.

The grid oracle is a floor on the available headroom, not a ceiling on DAHS. 72.29% is what a selector restricted to a coarse 4x4 partition of two state dimensions could achieve. DAHS reads 26 features (Section 4.2), so a finer partition of the state space can only raise the oracle. The gap is therefore a lower bound on the room available to a richer selector, and the honest statement is that *this projection* of the state space does not by itself justify selection.

Win rate is not the objective. A rule can win rarely and still be worth its slot if the states it wins are expensive ones, which is exactly why Table 3 screens on marginal contribution rather than on win rate — and the two orderings differ: COVERT wins 14.5% of decisions but carries a marginal contribution of 2.047, while EDD wins 6.8% and carries 0.007. The quantity that decides whether selection pays is composite cost, and Section 6.2 measures it. We did not compute a composite-cost oracle on the same 4x4 grid. What we can say from the test set is that the static champion on J is Always-COVERT, not Always-EEDD, and that DAHS beats Always-COVERT on 49 of 50 shifts (Section 6.2). Win-rate concentration on EEDD therefore overstates how little room there is on the objective.

Figure 1 is the state-space win-rate grid described above.

6.2. Main comparison

Rank the table by composite cost. Table 5 is the live comparison under the objective of Section 3.3 and causal periodic-review admission.

6.2.1. Two modelling defects that would invalidate a rule comparison

If the objective ignores priority weights, or if the dispatcher idles pickers on orders that have not arrived, a rule comparison does not measure rule quality.

Cause 1: the objective did not measure what the rules optimise. WSPT and ATC rank by w_o/p_o , using the priority weights of Section 3.1. An unweighted objective grades those rules against a criterion they were not designed for. Section 3.3 puts w_o into J , so rule and objective agree.

Cause 2: the dispatcher idled pickers on behalf of arrival-agnostic rules. Admitting every order that will arrive before the end of the current interval, and assigning start times $\max(\text{picker free}, a_o, t)$, reserves a picker for an order that is not yet in the building. FIFO, sorted by arrival, never triggers this; WSPT and ATC, which are arrival-agnostic, do. Section 3.4 admits only orders that have arrived by t .

All results in this section use that environment and the screened pool of Section 3.6.

Table 5. Default scenario, 50 test shifts, composite objective and causal admission. Ranked by composite cost, the quantity every learned method optimises. Mean arrived = 791 (the per-shift count is not constant; it is identical across methods on each seed). Dropped = 0. Paired 95% intervals are bootstrap percentile intervals of (method - DAHS) composite cost over the 50 aligned shifts, 10,000 resamples. The $\times$ column is a ratio of means; the bracket is that difference interval, not a confidence interval for the ratio. Tardiness is the KPI mean of censored lateness (tardiness_accounted): served orders at finish time, unserved orders at T , **without** the $+p_o$ the objective uses in J . The two quantities are not interchangeable.

Method	Composite				Thru.	Util.	Cost		
	cost	SFR	Spoil	Tardiness			Latency (ms)	vs DAHS	DAHS wins
greedy_mpc ($\tau = 1$, $M = 5$)	356.98	0.0590	0.0361	0.705	732.5	0.956	594	0.93× [-39.4, -12.8]	19
rolling_mpc ($\tau = 4$, $M = 5$)	363.42	0.0624	0.0381	0.748	732.4	0.956	670	0.95× [-28.1, -10.8]	15
DAHS	382.27	0.0669	0.0384	0.754	731.9	0.956	4.24	—	—
snapshot_xg ($\tau = 1$)	388.97	0.0651	0.0327	0.714	732.0	0.956	3.65	1.02× [-3.6, 18.2]	32

Method	Composite				Thru.	Util.	Latency (ms)	Cost	DAHS wins
	cost	SFR	Spoil	Tardiness				vs DAHS	
offline_fqi	397.64	0.0702	0.0452	0.779	732.0	0.956	3.26	1.04× [2.9, 29.8]	31
COVERT	455.21	0.0812	0.0830	0.788	732.1	0.956	<0.01	1.19× [60.9, 85.0]	49
LinUCB	552.39	0.0838	0.0672	0.751	731.2	0.956	—	1.45× [94.7, 264.2]	46
ATC ($k = 3.0$)	560.76	0.1075	0.0914	1.133	734.3	0.956	<0.01	1.47× [150.9, 206.6]	50
ppo_fair (un-tuned)	611.77	0.0924	0.0887	0.852	730.5	0.956	0.32	1.60× [114.0, 380.5]	50
EEDD	696.62	0.0915	0.0348	0.763	729.8	0.956	<0.01	1.82× [141.4, 528.8]	21
MDD	734.00	0.0970	0.0986	0.635	730.5	0.956	<0.01	1.92× [200.9, 540.9]	49
EDD	763.90	0.0985	0.1000	0.725	729.8	0.956	<0.01	2.00× [211.9, 594.5]	49
MS	790.67	0.1010	0.1024	0.760	729.1	0.956	<0.01	2.07× [225.1, 635.7]	49
WSPT	1216.54	0.0957	0.0591	5.387	743.3	0.955	<0.01	3.18× [753.4, 911.9]	50

Method	Composite						Cost		
	cost	SFR	Spoil	Tardiness	Thru.	Util.	Latency (ms)	vs DAHS	DAHS wins
FIFO	1486.82	0.1837	0.0720	1.963	730.2	0.956	<0.01	3.89× [895.5, 1342.4]	50
FEFO	1699.80	0.1971	0.0001	2.738	730.3	0.956	<0.01	4.45× [1077.5, 1584.8]	50

Wins are shifts on which DAHS is strictly cheaper. Under the confirmatory paired interval, every static rule, both teachers, `offline_fqi`, `LinUCB` and Table 5’s `ppo_fair` row reject equality with DAHS on composite cost. `snapshot_xgb` does not: the interval includes zero ($[-3.6, 18.2]$). Wilcoxon+ BH on the same 15 comparisons rejects `snapshot_xgb` ($p_{\text{adj}} = 0.029$; 32 wins, 14 losses, 4 ties, zeros discarded). On service-failure rate the snapshot pair is a null ($p_{\text{adj}} = 0.33$). We treat $\tau = 1$ as not demonstrably different from deployed DAHS on J under the confirmatory rule, and as a null on SFR.

The test block is a weak instrument wherever paired differences are frequently zero or tail-driven. Always-EEDD is bit-identical to DAHS on 22 of 50 shifts (21/7/22 wins-losses-ties; median paired difference 0). Under low load (Table 6) thirteen methods share $J = 9.82$. Against Always-EEDD the paired-difference SD is 705, so an 80% power minimum detectable mean effect on this $n = 50$ is about 285 cost units, or 75% of DAHS mean J ; a 4% effect has essentially no power. Against Always-COVERT the same 50 shifts detect a 4.7% mean effect at 80% power (49/50 DAHS wins). Against fitted Q the observed 4% gap has approximate paired- t power 0.58; the 80% minimum detectable effect is 5.2% of mean J . Nulls on this block are unsurprising when many shifts cannot distinguish the methods. They are not a demonstration that a 4% effect is absent.

Figure 2 is that forest plot.

Three facts about the live table:

1. **The teacher does not beat one-step lookahead**, and DAHS does not recover the teacher. Distillation is an amortisation of a *worse* scoring rule at $M = 5$, not of an oracle. Per-decision latency is the quantity DAHS wins (4.2 ms vs 670 ms for `rolling_mpc`; Section 6.12).
2. **Always-COVERT, not Always-EEDD, is the static to beat on cost.** Win rate on the Section 6.1 grid is the wrong proxy for the objective. Against COVERT, DAHS wins 49 of 50 shifts at 1.19 times; against EEDD the mean ratio is 1.82 times but DAHS is strictly cheaper on only 21 shifts, EEDD on 7, and they tie on 22. The EEDD mean is tail-driven (worst shift 5300 against DAHS 2440). DAHS also wins the median (44.2 against 57.7).

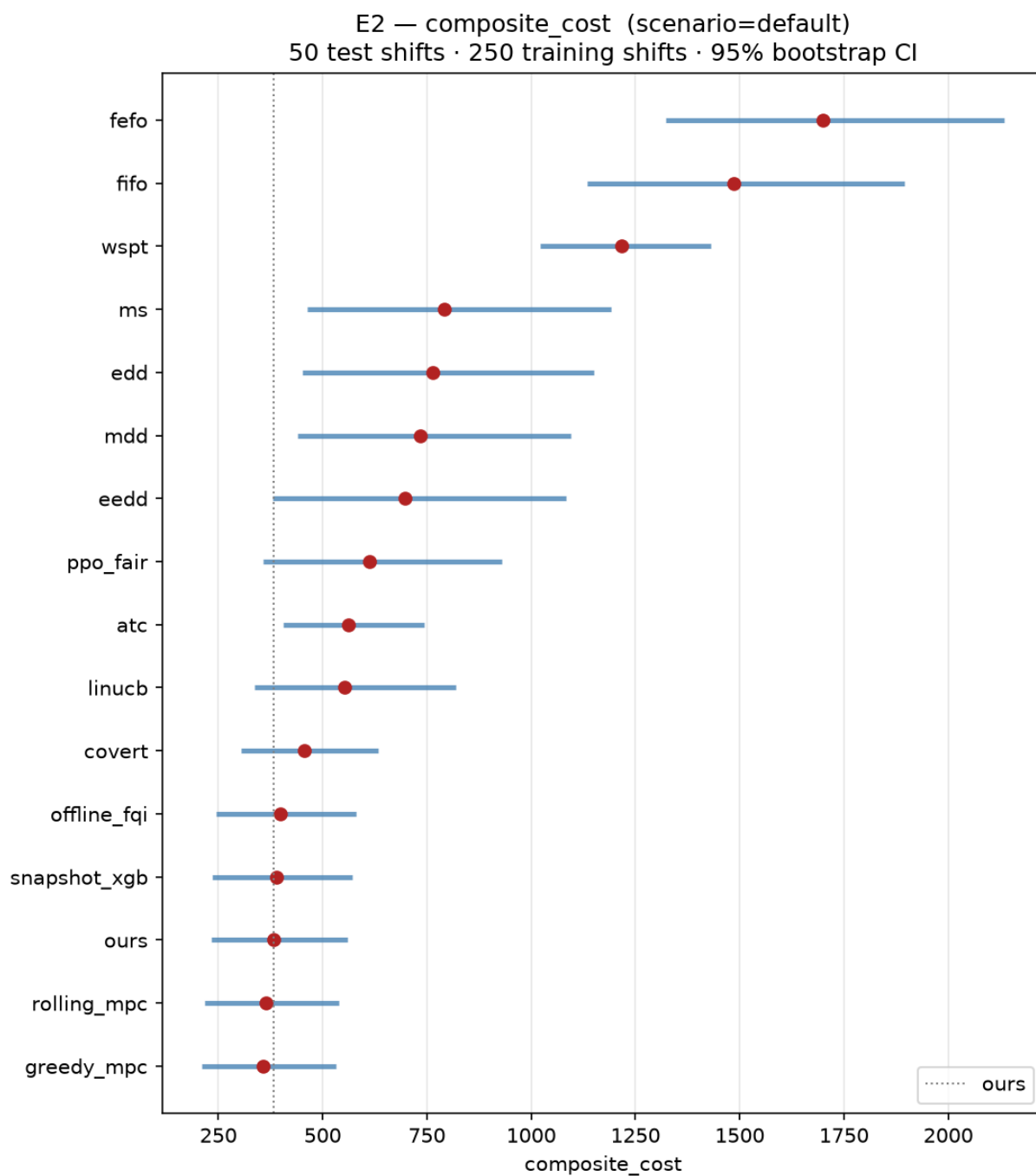


Figure 2: Default-scenario composite cost by method, 50 held-out shifts. Paired against DAHS.

3. **WSPT has the highest throughput** (743 vs FIFO 730) and every method sits at picker utilisation ≈ 0.956 . Cause 2 is the mechanism that would have produced idle pickers under non-causal admission.

Table 6. Composite cost and service-failure rate across four scenarios, 50 shifts each. Best static is the cheapest always-on rule in that scenario.

Scenario	DAHS J / SFR	Best static	Teachers (J)	FQI J	PPO J
default	382.27 / 0.0669	COVERT 455.21	greedy 356.98; rolling 363.42	397.64	611.77
high-load-perish	11430 / 0.542	WSPT 11171	greedy 11467; rolling 11341	11471	23226
balanced	17.73 / 0.00373	EEDD 17.62	greedy 17.92; rolling 17.33	18.50	35.65
low load	9.82 / 0.00327	COVERT (13-way tie at 9.82)	9.82	9.82	9.82

DAHS does not dominate across regimes. Under high-load-perishable WSPT is cheaper (paired cost difference -258 , interval $[-332, -187]$, BH-reject); Always-ATC is also cheaper than DAHS there ($J = 11,380$). Under balanced, EEDD is cheaper by 0.10 with a null test (interval $[-0.31, 0.00]$). Under low load thirteen methods, including DAHS and both teachers, return identical $J = 9.82$. The default-scenario ranking is therefore not a licence to write that selection beats any single rule in every operating region.

6.2.2. Boundary conditions: what the selector actually does under saturation

Calling a lost cell “a saturation effect” is an attribution, not an analysis. There are two candidate explanations and they have different remedies.

The first is that the selector stops selecting. We measure this as the **exponentiated entropy of the deployed-rule distribution**, which equals $|\mathcal{H}|$ when the selector spreads across the pool and 1.0 when it has collapsed. On 1,600 default epochs the EEDD share is 0.849 and the COVERT share is 0.138 (switch rate 0.039). Under high-load-perishable those shares invert: COVERT 0.828, EEDD 0.173. Under balanced and low load the selector is effectively Always-EEDD (shares 0.995 and 0.999). Collapse onto a specialist is the mechanism in the light-load cells; a switch into COVERT is the mechanism under perishable saturation.

The second is that the guardrail binds. The minimum dwell holds a rule for $T_{\min}$ epochs, and under saturation the queue state changes fastest — exactly when a stale rule is most costly. We measure this as the **blocked-switch rate**: the share of epochs at which the ranker’s arg-max differed from the deployed rule *because the dwell was still active*. A $T_{\min}$ sweep *within* high-load-perishable finds the cost-minimising dwell tied at the deployed $T_{\min} = 2$ and at $T_{\min} = 3$

Sample efficiency: DAHS vs the snapshot ranker and analytic MPC, by training-shift budget

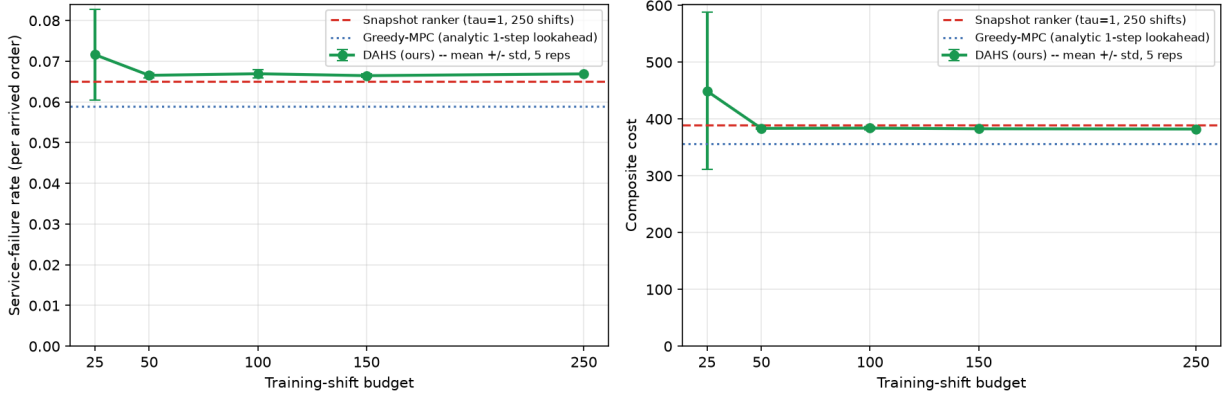


Figure 3: Sample efficiency. DAHS mean composite cost versus the number of simulated training shifts, five independent labelling-and-training replicates except $n = 250$ (one replicate, the deployed model). Source `figures/data_efficiency/data_efficiency_curve.png`.

(both 11429.77); $T_{\min} \in \{0, 1\}$ is 11429.85 and $T_{\min} = 4$ is 11432.15. The guardrail is not the boundary condition in that scenario.

6.3. Sample efficiency

Figure 3.

Table 7. DAHS data-efficiency, composite cost. Budgets {25, 50, 100, 150, 250}; five replicates except $n = 250$ (one replicate by design).

n shifts	Replicates	Mean J	Mean SFR	Notes
25	5	449.17	0.0716	One of five collapsed to Always-EEDD ($J = 696.62$)
50	5	383.46	0.0665	Already at the deployed level
100	5	383.88	0.0669	
150	5	382.80	0.0665	
250	1	382.27	0.0669	Deployed model

The selector saturates by 50 shifts. The $n = 25$ collapse is a real failure mode — not every short corpus yields a working ranker — but it is one replicate in five, not the typical outcome. Fitted Q-iteration on the same budgets is

slower to saturate on the pre-admit evaluation (mean J : 580, 483, 456, 412, 397 at $n = 25, 50, 100, 150, 250$; Section 6.10). Those FQI fits were not retrained after terminal admit; Table 5’s live FQI row is 397.64.

6.4. Rollout horizon, and the number of continuations

The truncation sketch predicts that truncation bias shrinks as τ grows. The operational test is a sweep over $\tau \in \{1, 2, 3, 4\}$, each arm labelled at $M = 20$ and used to train an otherwise identical ranker, evaluated on the same 50 test shifts. Cross-validated soft cross-entropy on the training labels is not the deployment criterion; we report it alongside J because it is what the ranker actually fits.

Table 8. Rollout-horizon sweep. Cost intervals are 95% bootstrap intervals of the 50-shift mean. Those intervals are wide because they are marginal means, not the confirmatory paired test of Section 5. $\tau = 1$ is cost-identical to snapshot_xgb on non-timing KPIs (latency columns differ); $\tau = 4$ is cost-identical to Table 5’s DAHS on non-timing KPIs.

τ	J	95% CI of	SFR	Median label	Test rows kept
		mean		entropy	
1	388.97	[236.3, 567.7]	0.0651	0.693	915 / 1600
2	373.16	[225.7, 546.6]	0.0633	0.649	1273 / 1600
3	374.23	[226.0, 549.0]	0.0637	0.638	1444 / 1600
4	382.27	[233.3, 556.1]	0.0669	0.638	1525 / 1600

Every *marginal* mean interval overlaps every other; those intervals are the wrong test for a τ claim (they would also fail to separate DAHS from Always-COVERT). The confirmatory paired interval of $(\tau - 4)$ on the same 50 shifts is: $\tau = 1 + 6.71 [-3.63, 18.17]$ (includes 0); $\tau = 2 - 9.11 [-17.61, -1.35]$ (excludes 0); $\tau = 3 - 8.04 [-16.90, 0.28]$ (includes 0). $\tau = 2$ is cheaper than the deployed $\tau = 4$ model on mean J . The same 50 shifts split 21 cheaper / 20 worse / 9 ties; the median paired difference is 0; Wilcoxon signed-rank $p = 0.221$. That is a mean-driven interval, the same tail pattern disclosed for Always-EEDD (21/7/22). $\tau = 3$ includes 0 on the interval; its unadjusted Wilcoxon p is 0.0495 (BH $m = 3$, $p_{\text{adj}} = 0.074$). On SFR, $\tau = 1$ versus $\tau = 4$ includes 0 ($p_{\text{adj}} = 0.33$). The deployed ranker is $\tau = 4$ because the labels were frozen there. We do not read Table 8 as confirming an interior τ^* . Figure 4.

The companion sweep is $M \in \{1, 5, 10, 20, 40\}$. All five cells completed. Figure 5 plots composite cost against M .

Table 9. Number of continuations. $M = 20$ is the deployed model. Entropy is in-band at every M . $M = 1$ has identically zero rollout SE, so the fraction of labels with separation below one SE is zero by construction. $M = 1$ post-calibration ECE 0.067 misses the 0.05 bar.

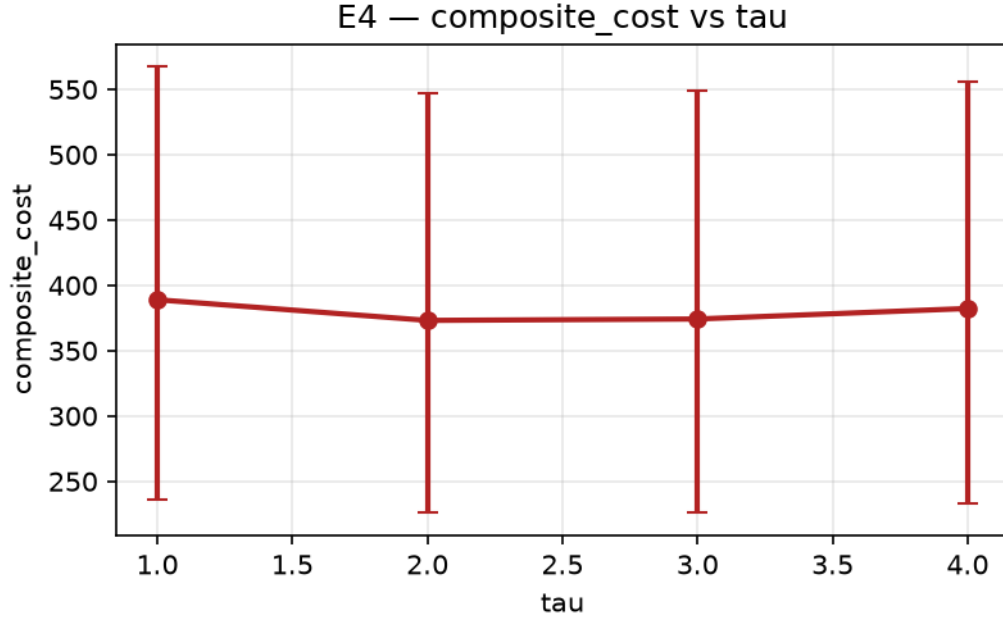


Figure 4: Composite cost versus rollout horizon τ . Source `results/E4/tau_summary.parquet`.

M	J	SFR	Median entropy	frac <1 SE	Test kept	ECE pre to post
1	381.08	0.0664	0.641	0	1150 / 1600	0.077 -> 0.068
5	381.28	0.0660	0.644	0.567	1384 / 1600	0.123 -> 0.034
10	383.25	0.0670	0.649	0.455	1457 / 1600	0.155 -> 0.026
20	382.27	0.0669	0.638	0.334	1525 / 1600	0.170 -> 0.021
40	383.63	0.0666	0.652	0.215	1526 / 1600	0.189 -> 0.018

Paired against $M = 20$, $M = 1$ differs by -1.18 in J (95% CI $[-7.83, 5.18]$, Wilcoxon $p = 0.91$); $M = 5$, $M = 10$ and $M = 40$ likewise include zero ($M = 40$: $+1.36$, $[-3.05, 5.99]$, $p = 0.35$). The five-cell span on mean J is about 0.7%. Multi-sample labels improve calibration ECE and keep more test rows through the ambiguity filter. They do not improve deployed cost. Why $M > 1$ remains the argument of Section 4.3: a single continuation is a hindsight-optimal path, not an estimate of expected cost. That is not a claim that $M = 20$ is an accuracy lever.

6.5. Robustness across untuned configurations

36

Figure 6 is that heat map.

Five methods are frozen across the grid: DAHS, greedy_mpc, snapshot_xgb, Always-EEDD and Always-COVERT. Teachers replan with each cell’s true arrival rate and tightness; DAHS and snapshot_xgb are the default-trained rankers with no retraining. greedy_mpc wins 8 of 12 cells and EEDD wins the four light-load default/loose cells. DAHS wins none, and adding Always-COVERT does not change that: COVERT also wins none. The `arr1.65_default` cell

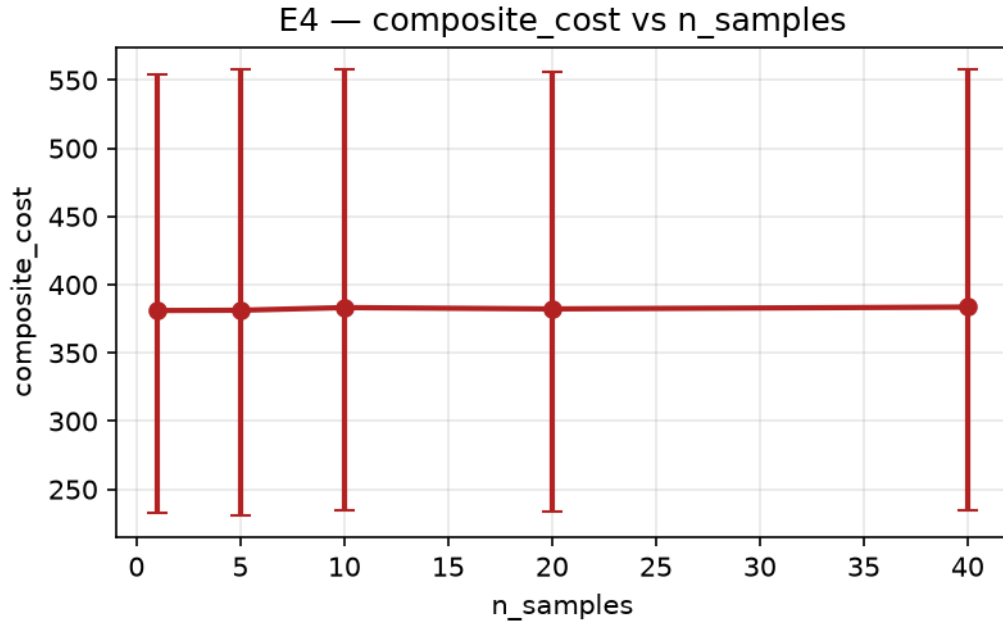


Figure 5: Composite cost versus number of continuations M . Source `results/E4/n_samples_summary.parquet`.

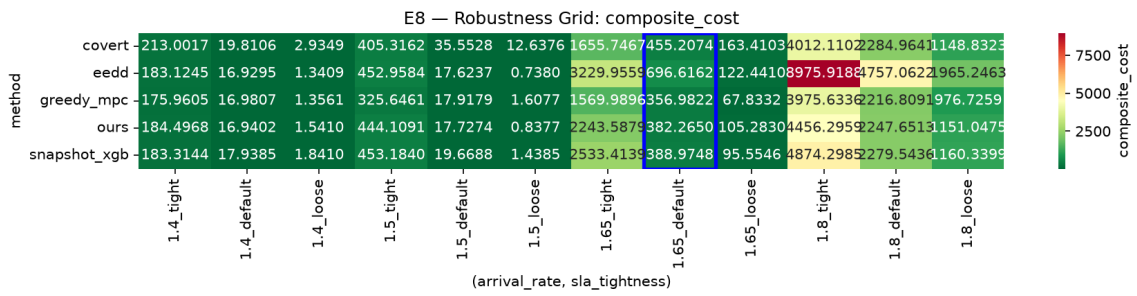


Figure 6: Robustness grid across 12 untuned configurations (4 arrival rates $\times$ 3 SLA tightnesses). Heat map of composite cost. The outlined cell is the Table 5 default and matches it to machine precision.

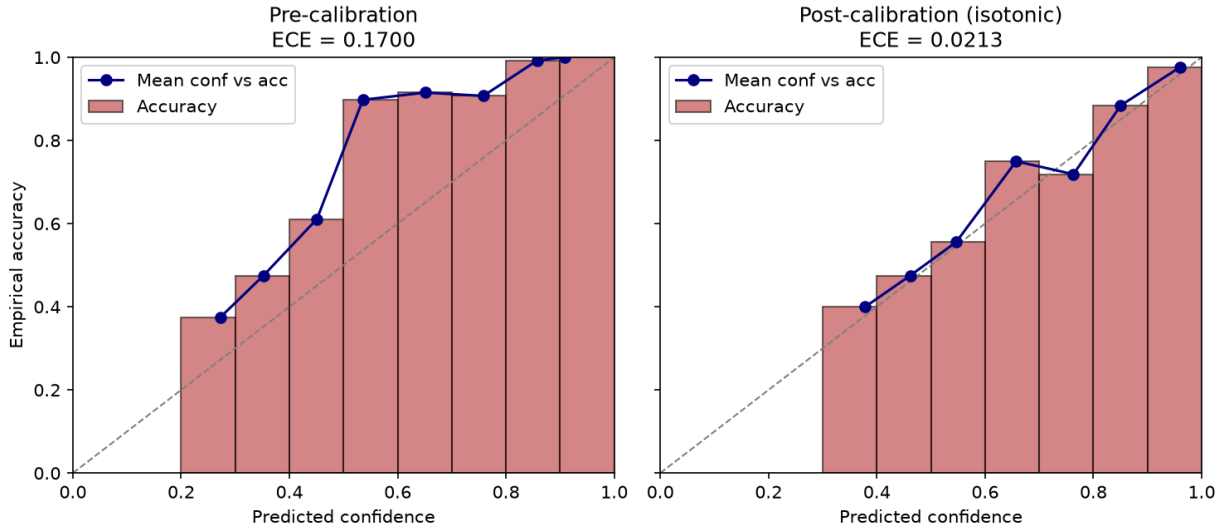


Figure 7: Reliability diagrams before and after isotonic calibration.

6.6. Calibration and interpretability

Isotonic calibration on a 20% held-out shift split improves expected calibration error from 0.1700 to 0.0213 (design target 0.05) and Brier score from 0.1730 to 0.1240, and *degrades* soft cross-entropy from 0.828 to 2.358. EDD never wins a label on the calibration split and is passed through uncalibrated. The reliability diagrams are Figure 7.

Global SHAP values (Lundberg and Lee 2017), computed as a *diagnostic* on labelled test states (`data/test.parquet`), on the 38-dimensional ranker input (26-feature observation plus 12 regime posterior coordinates) rank `queue_length` (0.370), `queue_length_lag_1` (0.152), `mean_slack_minutes` (0.120), `interval_index_in_shift` (0.053) and `queue_length_lag_2` (0.052). The sixth coordinate is a regime one-hot (`regime_post_9`, 0.040). Congestion and slack dominate; the product-clock features do not. The remaining 33 of 38 ranker coordinates are not earning their dimensionality on cost at a detectable margin on this test set (Section 6.8). That is consistent with Section 3.5: perishability binds, but the customer clock is the more frequent pivot, and EEDD already reads both clocks inside the action. Figure 8.

6.7. Real-data grounding

Fitting, not validating. Inter-arrivals are fitted to a public trace. Processing time and shelf life are design parameters. Appendix C reports the candidate families, the AIC table, and the selected family. Inter-arrivals are lognormal (Olist, $n = 98,241$); the exponential alternative is worse by $\Delta\text{AIC} = 34,502$. Customer windows are the closest shape match (KS $D = 0.039$). Processing time is not fitted: the trace’s purchase-to-approval delay is not pick time. Perishable fraction 0.20 is a design choice (Olist food/drink share is 0.0099). The operating arrival rate 1.65 / min is an operating point, not a fit. Figure 9.

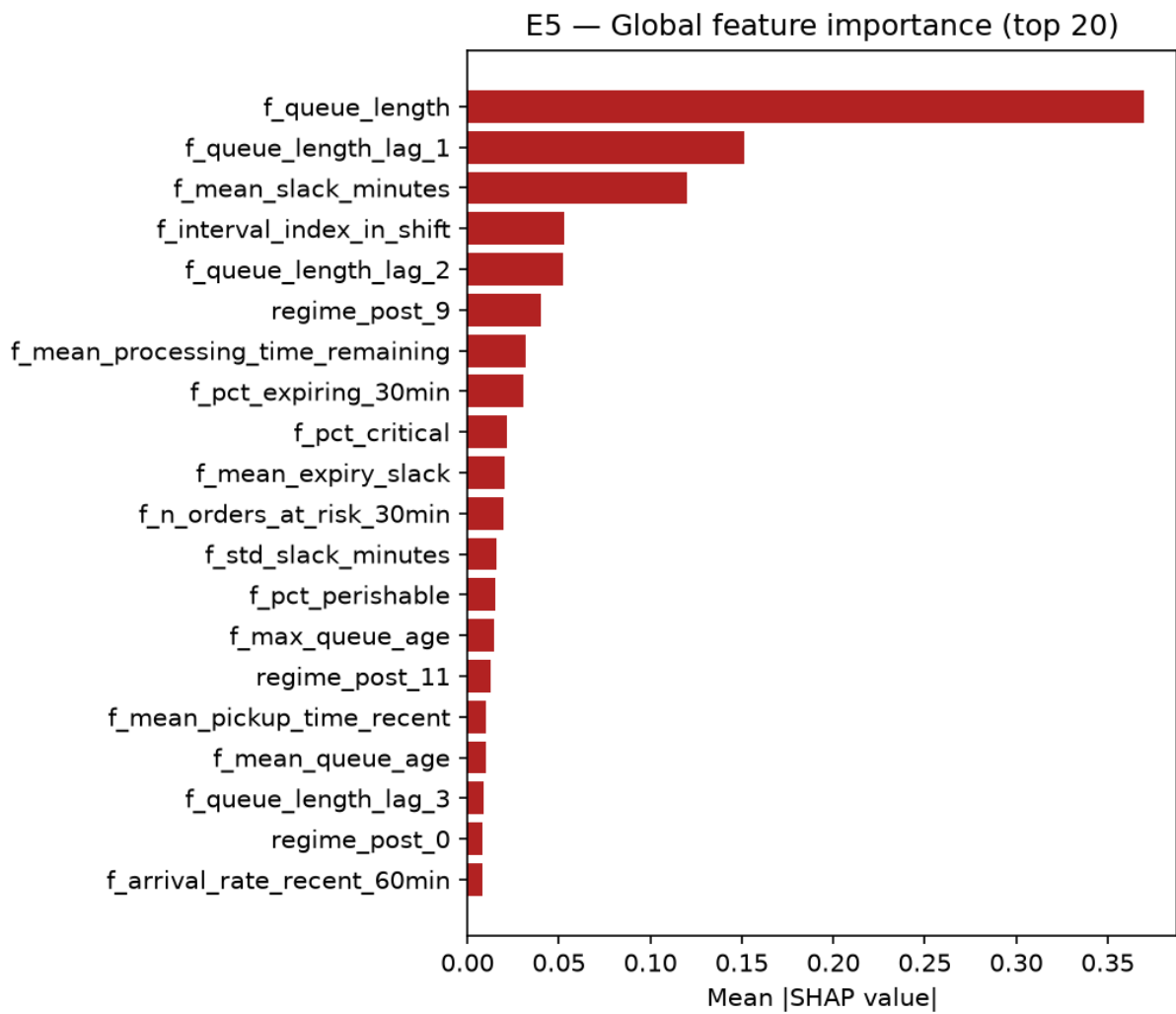


Figure 8: Global SHAP feature importance for the ranker.

Deliverable A — Olist trace vs. simulator draws (distribution shape, mean-normalized)

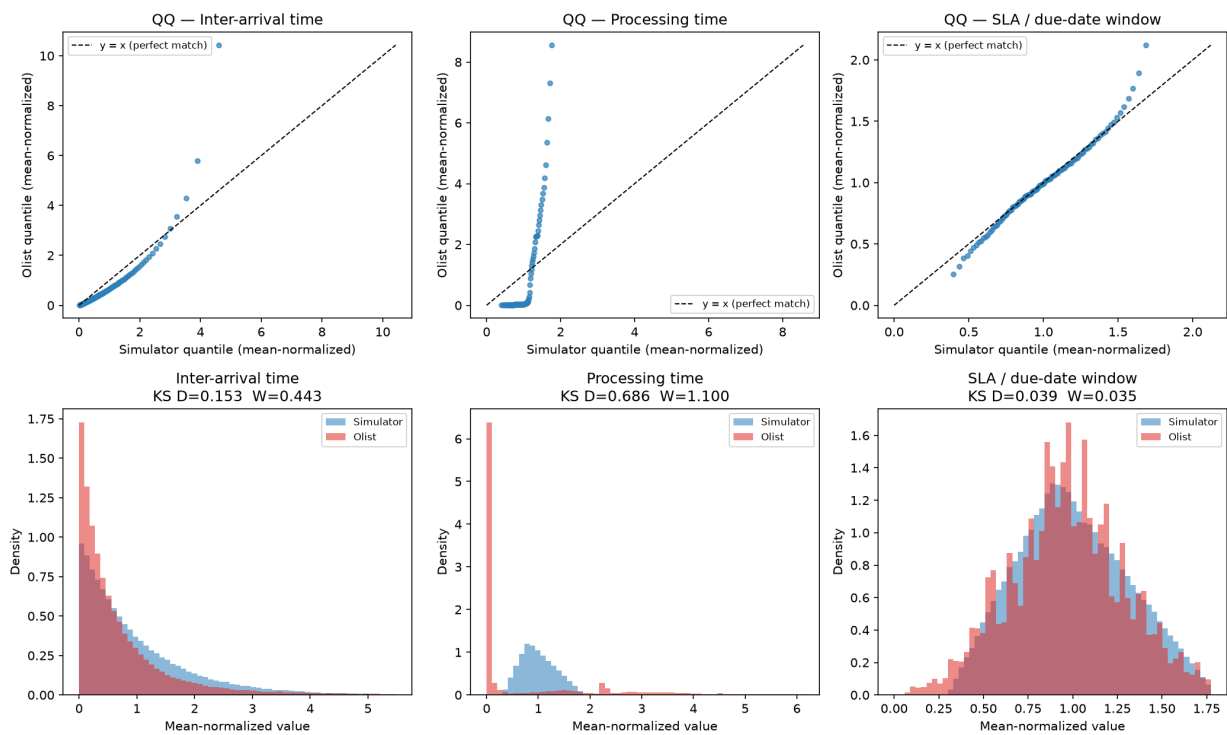


Figure 9: Simulator input distributions against the Olist order trace (mean-normalised).

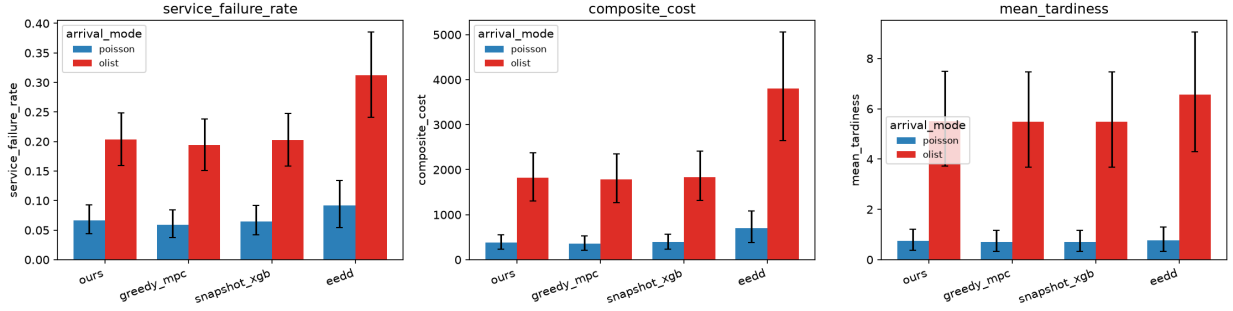


Figure 10: Method KPIs under Poisson against empirical-Olist bursty arrivals (frozen rankers).

A frozen-ranker replay under empirical Olist arrivals (burstiness CV 2.68, against Poisson CV 1.00) preserves the ranking: greedy_mpc 1784, DAHS 1816, snapshot_xgb 1840, Always-EEDD 3807. Absolute costs inflate; the teacher still leads and EEDD still trails. Figure 10.

6.8. Ablations

Table 10. Retrain and inference ablations versus DAHS, 50 paired shifts. Composite cost is the column that decides. The confirmatory functional is the mean (Section 5). no_regime is worse than DAHS on mean J (+2.14, [0.04, 4.61], excludes 0). That lower bound is 0.04 on a cost of 382, and Wilcoxon+BH on the eight-arm family in e3_summary.parquet (including random_ambiguity_filter, which is per-shift identical to DAHS) does not reject ($p_{\text{adj}} = 0.34$). We report a mean-cost difference of negligible operational size, not a finding that the regime layer is load-bearing. The other displayed retrain arms include 0. random_ambiguity_filter is omitted from the displayed rows; the reported p_{adj} values are the 8-arm corrections. A top5_features retrain that selected coordinates from a test-set SHAP ranking is omitted from this table; it is a diagnostic in the paragraph below, not a held-out ablation. Non-rejection is not an equivalence test. The training wall-clock to convergence is reported for retrain arms; inference-only arms do not retrain.

Ablation	J vs DAHS		p_{adj}	SFR	Train wall (s)
	J	[95% CI]			
hard_labels	383.28	+1.01 [-5.52, 7.13]	0.83	0.0665	1302
no_calibration	382.31	+0.04 [-5.76, 4.89]	0.83	0.0665	—
no_regime	384.41	+2.14 [0.04, 4.61]	0.34	0.0671	1124

<i>J</i> vs DAHS					
Ablation	<i>J</i>	[95% CI]	p_{adj}	SFR	Train wall (s)
no_switching_controller	381.27	-0.99 [-4.24, 1.96]	0.83	0.0667	—
single_sample_rollout ($M = 1$)	381.08	-1.18 [-7.83, 5.18]	1.00	0.0664	—
DAHS	382.27	—	—	0.0669	—

The hard-label comparison is a null on both composite cost and service-failure rate. Soft labels remain the default because they are the deployed configuration, not because they outperform one-hot arg-max. Removing the switching controller slightly *improves* point-estimate cost; the wrapper is a deployability guardrail paid in KPI, not a performance component. A `top5_features` retrain that selected five coordinates from a test-set SHAP ranking scored $J = 383.99$ (+1.72, [-3.44, 6.59]). That arm is not in Table 10. It is a diagnostic with selection on the evaluation corpus, not a confirmation-set protocol.

A single-sample rollout ablation ($M = 1$ relabel and retrain) is the $M = 1$ cell of Table 9 and the `single_sample_rollout` row of Table 10 (cost-identical parquets; latency columns differ). It matches $M = 20$ on J . The $\tau = 1$ snapshot is Table 8.

6.9. On the PPO baseline

Untuned PPO at 8000 timesteps (`ppo_baseline`, `n_steps = 64`) collapses to a constant EEDD policy: its 50-shift cost is 696.616200, identical to Always-EEDD to machine precision. That is not the Table 5 `ppo_fair` row ($J = 611.77$), which is a separately trained stock Stable-Baselines3 policy with a full KPI record. Sensitivity analysis: a 12-configuration grid was scored on the 50 test shifts (`experiments/rl_sensitivity.py`). Eight of the twelve cells later score the same calibration cost 587.36, so the calibration split barely discriminates and cannot repair a test-first search. We do not treat “the same cell wins both rankings” as evidence that selection was on calibration. Table 5 keeps the untuned `ppo_fair` row, which was declared before this grid. The grid is a diagnostic of how much normalisation buys, not a replacement baseline.

Table 11. PPO sensitivity grid. Against the Always-EEDD *collapse* row ($J = 696.62$), `gap_closed_fraction` = (collapse – best)/(collapse – DAHS) = 0.783. Against Table 5’s named untuned `ppo_fair` row ($J = 611.77$) the same best cell closes 0.70. We do not call both denominators “untuned.” Factor spreads: normalisation 313.5; `n_steps` 22.5; γ , GAE λ and entropy coefficient 0.

Configuration	<i>J</i>
8 of 12 (including the untuned default)	696.62

Configuration	J
n_steps=256	674.14
obs-norm only	541.51
obs-norm and reward-norm	450.45
reward-norm only	763.90 (Always-EDD)

Tuning closes a substantial share of the gap. The reading that PPO’s deficit is structural rather than budgetary is withdrawn. The winner (norm(obs=True, rew=True), $J = 450.45$) is a sensitivity cell scored on the test block, not a baseline. Eight of twelve calibration costs tie at 587.36, so a calibration-first pass would barely discriminate. The 0.783 figure is the collapse-to-DAHS fraction; 0.70 is the ppo_fair-to-DAHS fraction. Table 5 keeps the untuned ppo_fair row, which is the policy with a full KPI record; the grid cell is reported here as a diagnostic.

6.10. On the offline reinforcement-learning baseline

Action coverage. The behaviour policy is random. Round-robin over the pool would not be: the interval index is an observed feature, so $a = t \bmod |\mathcal{H}|$ is a deterministic function of the observation and conditional coverage would be one action per state by construction. On 8,000 training states the effective number of actions is 5.999 of 6 overall and 5.995 of 6 on the breach-prone quartile; the interval-conditional mean is 5.937. Coverage is adequate: the offline-RL deficit cannot be attributed to unsupported actions in the hard region.

After an observe-once logger (calling observe() twice per interval zeros n_arrivals_last_interval on 31 of 32 states), DAHS beats fitted Q on composite cost, 382.27 versus 397.64 (ratio 1.04, paired interval [2.90, 29.83], confirmatory interval excludes 0). The margin is about 4%. It is the only learned-baseline comparison in Table 5 that favours the selector. Section 8 records that it is not a demonstration about value learning. Figure 11.

Frozen across the four scenarios and the twelve-cell grid, FQI tracks DAHS rather than collapsing. Under high-load-perishable both sit near 11,430–11,471 while PPO reaches 23,226 and Always-EEDD 24,065. Under default, FQI is the fifth method in Table 5, between snapshot_xgb and Always-COVERT.

6.11. Model misspecification: labelling in one world, deploying in another

The experiment labels once under the nominal simulator and evaluates frozen controllers — DAHS, rolling_mpc, offline_fqi, Always-EEDD, Always-COVERT — under perturbed dynamics: arrival rate $\times \{0.8, 0.9, 1.0, 1.1, 1.25\}$, processing time on the same grid, SLA $\times \{0.8, 1.0, 1.25\}$, shelf life $\times \{0.8, 1.0, 1.25\}$, and picker count $\Delta \in \{-2, -1, 0, 1, 2\}$. The online teacher is pinned to the *nominal* model, so it replans with a wrong model rather than with the truth.

Table 12. Mean relative-degradation slope of composite cost against the perturbation, averaged over axes.

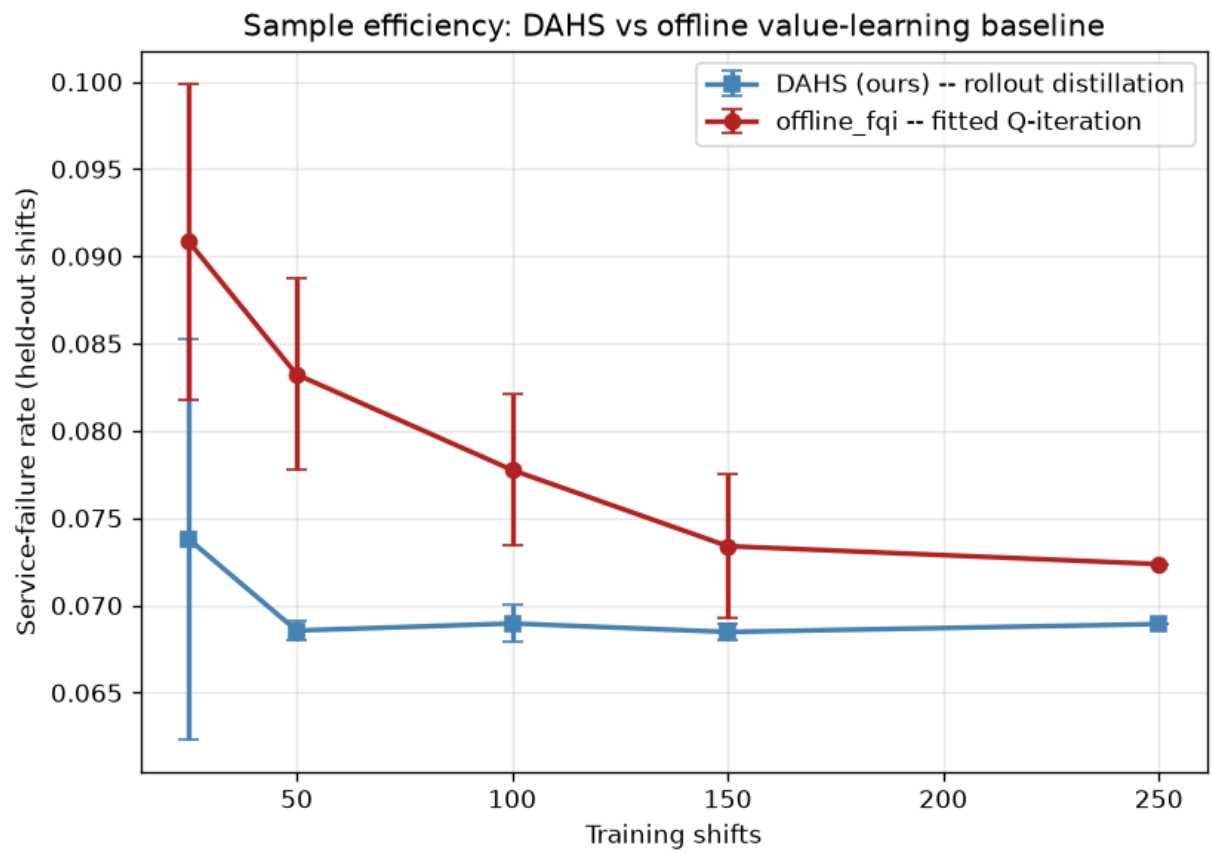


Figure 11: Sample efficiency: DAHS versus fitted Q-iteration at matched shift budgets. The FQI curve is the pre-admit evaluation ($|A| = 767$ at $n = 250$, $J = 396.80$); Table 5’s live FQI row is 397.64.

Method	Mean slope	Rank (slower degradation first)
COVERT	18.7	most robust (model-free)
offline_fqi	21.9	
DAHS	22.7	
rolling_mpc	23.6	
EEDD	27.7	least robust

DAHS is not the most robust model-based method; FQI’s mean slope is slightly shallower. Always-COVERT, which carries no model, degrades slowest. Always-EEDD degrades fastest on load, capacity and processing time. On the SLA axis DAHS has the *steepest* slope of the five. Shelf-life slopes are near zero.

This experiment does not retrain at $\tau \in \{1, 2, 3, 4\}$ as ε grows, so it does not test whether an operationally best horizon would shorten. We report the slopes and that the τ – ε interaction is unmeasured.

6.12. Computational cost, and scaling in the size of the rule pool

6.12.1. Offline cost

Walking each shift forward once and branching at each epoch (Section 4.3) costs

$$N + N \cdot |\mathcal{H}| \cdot M \cdot \tau \quad \text{interval-steps per shift,}$$

linear in N : the labeller walks each shift once and branches at each epoch, rather than replaying each shift from $t = 0$ for every candidate. For the deployed setup — $N = 32$, $|\mathcal{H}| = 6$, $\tau = 4$, $M = 20$, 250 training shifts — labelling consumed 4,401,600 interval-steps (3,668,000 train, 733,600 test) in 758 s train and 164 s test wall-clock on the campaign machine (Intel Core i9-14900K, 24 cores / 32 threads, 64 GB, Windows 11, Python 3.12.10). Rule calibration, which sweeps k under $M = 5$, consumed 2,966,400 additional interval-steps. A separate single-thread `compute_budget` measure pass on three shifts recorded 389.3 interval-steps per second (2.75 h single-core for the 3,848,000-step corpus formula used by that driver). Labelling wall-clock above is the labeller’s own multi-core measurement and remains the campaign figure.

6.12.2. Online cost, and per-decision inference latency

Table 13. Mean per-decision inference latency, 50 default shifts. The p95 column is the mean, across shifts, of each shift’s per-decision p95 (so DAHS can have mean 4.24 ms and p95 4.25 ms). It is not a pooled p95 over all decisions.

Method	Mean (ms)	p95 (ms)	Wall-clock s / shift
DAHS	4.24	4.25	0.163

Method	Mean (ms)	p95 (ms)	Wall-clock s / shift
snapshot_xgb	3.65	3.52	0.143
offline_fqi	3.26	5.12	0.130
ppo_fair	0.32	0.46	0.024
greedy_mpc	594	859	19.0
rolling_mpc	670	941	21.5
COVERT / EEDD / FIFO	<1e-3	<1e-3	0.01

DAHS is 158 times faster per decision than `rolling_mpc` (4.24 ms vs 670 ms) and 140 times faster than `greedy_mpc`. Static rules remain three orders of magnitude faster still. The amortisation claim is this ratio, on named hardware, not an argument that a forward pass is faster than a tree of simulations. At a 15-minute review interval, 670 ms is 0.07% of the epoch, so the latency product is real and operationally unused unless a tighter review is required.

Labelling the $M = 20$, $\tau = 4$ corpus took 922 s (15.4 min) of wall-clock. Ranker retraining on the same machine takes 15–27 minutes (Table 10: 897–1302 s for the retrain arms). At $21.5 - 0.16 = 21.3$ extra seconds per shift of online lookahead, labelling alone is repaid after about 43 shifts; labelling plus retraining is repaid after about 90–120 shifts. Both figures are hardware-specific and ignore that the teacher is also the better policy.

6.12.3. *Scaling in $|\mathcal{H}|$*

Labelling cost is linear in $|\mathcal{H}|$ under uniform allocation of M continuations to every rule. Two mitigations are discussed.

Adaptive sample allocation (successive halving) discards clearly dominated rules part-way through the M continuations and spends the remaining budget on the survivors. Hierarchical selection first screens a cheap one-step score and only then rolls the shortlist to depth τ . Successive halving is implemented in the labeller as `costs_at_epoch_successive_halving` and was measured as a compute-budget diagnostic; production labelling (`label_one_shift`) never calls it and stays on uniform `costs_at_epoch`. A 50-shift diagnostic at the deployed (M, τ) on the **nine-candidate screen set** (not the six-rule deployed pool) produced arg-max agreement 0.856 against uniform allocation, mean label KL 0.185, and a 1.1% step saving (`successive_halving.json`). That KL is between untempered softmaxes of the two cost vectors, not the production tempered labels $T(s) = \beta \hat{\sigma}(s)$. The verdict is unsuccessful, so production labels stay uniform. Hierarchical selection is described here and is not implemented. We did not retrain DAHS at pool sizes 2, 4 and 8. The statement we can support is the complexity: uniform labelling is $\Theta(N|\mathcal{H}|M\tau)$ interval-steps per shift; successive halving replaces $|\mathcal{H}|M$ with $|\mathcal{H}|\log M$ in the usual approximation when gaps are large relative to rollout SE. At $M = 20$, 33% of training epochs have a best-versus-second gap below one pooled SE (Section 4.3), which is exactly the regime in which early discarding is least safe.

7. Discussion

The campaign supports a smaller claim than the one we started with, and that is the claim we are willing to defend.

A practitioner who already has a resettable simulator and who can afford 670 ms per decision should run the one-step teacher. A practitioner who needs a millisecond decision can distil it, and should not expect the student to catch the teacher or to win the transfer grid. There is no third branch in which DAHS is the policy to run. If the simulator does not exist, these labels cannot be generated.

Two comparisons that looked structural are not. PPO’s deficit was a missing normalisation wrapper: the collapse-to-DAHS `gap_closed_fraction` is 0.783; the `ppo_fair`-to-DAHS fraction is 0.70. Round-robin FQI coverage would be degenerate by construction; under a random behaviour policy coverage is adequate and a 4% cost gap remains. Both baselines belong in the table. Neither is a demonstration that value-based or policy-gradient methods cannot do this task. PPO is not a matched simulation-step budget. The paper does not rest on the 4% FQI gap.

Perishability binds at this horizon (Table 1) at the fraction we ran. FEFO drives spoilage essentially to zero at catastrophic cost (Table 5). EEDD is the rule that reads both clocks. That does not imply that a selector over those rules will win every regime: WSPT wins high-load-perishable, and Always-EEDD is statistically tied with DAHS under balanced load.

8. Limitations

8.1. Shared-simulator circularity

Every method in Table 5, teachers included, is labelled or trained in the same simulator it is evaluated in. The comparison of training signals is internally valid. It is not a claim about transfer to a physical warehouse. Section 6.11 perturbs the evaluation dynamics while keeping the labels nominal; that is a start, not a substitute for a plant. A practitioner without a trustworthy simulator should prefer value learning from logs (Section 2.4).

8.2. A small heuristic pool

Nine candidates were screened and six retained. Adaptive sample allocation (successive halving) is implemented in the labeller; hierarchical selection is described in Section 6.12 and is not implemented. We did not measure accuracy as a function of pool size. The labelling cost grows with $|\mathcal{H}|$. A larger, generated pool — genetic programming of the low-level rules (Branke et al. 2016; Nguyen et al. 2017) followed by the same selector — is a different paper. EEDD’s 65% win rate already warns that a small hand-designed pool can concentrate.

8.3. A single warehouse setting

One layout, ten pickers, one-order tours, exogenous routing, Poisson default arrivals. Appendix C fits inter-arrival shape to Olist; processing time and shelf life are design parameters. The perishable fraction 0.20 is about twenty times the food/drink share in that trace (0.0099). Nothing here is a claim about multi-block warehouses, batching, or picker routing. Section 2.1 stated that restriction before any number. The method name DAHS is historical; Section 3 does not model disruptions.

8.4. No online adaptation after deployment

The ranker is frozen. Regime posteriors are computed from the fitted mixture, not updated from live outcomes. A shift that drifts away from the training mixture gets the wrong specialist. The misspecification grid (Section 6.11) is the evidence we have that this matters, and Always-COVERT’s shallower slope is a reminder that a dumb static rule does not carry a wrong model.

Further limitations, stated so they are not rediscovered as objections.

Partial observability. The aliasing rate among ϕ -identical training states that disagree on the cost-minimising rule is 0.145 on 1,927 pairs, with mean regret 0.30 and an estimated 3.7% share of the achievable benefit available from rule selection. That number is an upper bound on the part of the residual regret attributable to ϕ , not a reason to treat ϕ as a state.

Regime order. $K^* = 12$ is the edge of the swept grid. BIC is still falling (−168,901 at $K = 8$, −222,525 at $K = 10$, −240,139 at $K = 12$). Mean ARI is 0.970, so the clustering is reproducible; the model order is not identified. We did not widen the grid, because doing so would retrain Stage 3 onward.

A pool of six is what a dispatcher already has. Nine candidates were considered and six retained (Section 3.6). Adding generated rules would change the labelling budget and the action-coverage comparison, and we did not do it.

The FQI comparison is a 4% cost gap after a logger correction. It is not a demonstration that bootstrapped value learning cannot close that gap with a different model class or a different behaviour policy.

9. Conclusion

We studied a selection hyper-heuristic for periodic-review warehouse dispatching under two deadline clocks, trained by offline truncated rollouts of a screened rule pool, on one warehouse simulator. The training mechanism is not new. The question was whether the distilled ranker recovers the online teacher on that simulator.

It does not. On the default corpus, online truncated lookahead is cheaper (357–363) than the distilled ranker (382). Always-COVERT is the static champion (455; 49/50 shifts). Fitted Q is about 4% worse; that gap is not a result about value learning. PPO is not a matched simulation-step budget; untuned PPO costs 612, or 450 after observation and reward normalisation. Single-sample labels match the deployed $M = 20$ configuration on composite cost. A one-step

($\tau = 1$) label is a null on mean J under the paired interval. A two-step label is cheaper than deployed $\tau = 4$ on the mean (-9.11 , $[-17.61, -1.35]$; 21/20/9, median 0), with no median support. On the 12-cell robustness grid DAHS wins none of twelve cells among five frozen methods. Distillation is a 4.24 ms decision that spends 5–7% more than a 670 ms teacher; that 670 ms is 0.07% of a 15-minute epoch.

The most useful extension is the one the dynamic-dispatching literature already runs: replace hard truncation with a learned value tail (Ulmer et al. 2019; Goodson et al. 2017), and let the horizon be state-dependent. That construction would likely permit a shorter τ , which is attractive here because Table 8 says $\tau = 2$ already beats deployed $\tau = 4$ on mean J . Two other extensions follow from the campaign: widen the regime grid until K^* is interior, or drop the mixture; and test the null on a second environment. One simulator cannot support a claim about training signals in general.

10. Appendix A. State features, their provenance, and their redundancy

10.1. Where each feature comes from

This appendix lists every feature with its group and the source or design rationale. The table is generated directly from `simulation.state_extractor.FEATURE_PROVENANCE` by `experiments/feature_analysis.py`, so the manuscript and the deployed feature map cannot drift apart.

The observation is $\phi(S_t) \in \mathbb{R}^{26}$. It is an observation, not a state (Section 3.2).

Feature	Rationale
<code>queue_length</code>	Standard congestion state; de Koster et al. (2007).
<code>mean_queue_age</code>	Waiting-time proxy; distinguishes fresh from stale backlog.
<code>max_queue_age</code>	Tail of the waiting-time distribution — starvation detector.
<code>pct_critical</code>	Share of queue within 30 min of its due time.
<code>pct_perishable</code>	Share of the queue that carries a product clock. The expiry-rule mask reads this; on the screened pool the mask is a no-op.
<code>n_arrivals_last_interval</code>	Short-run demand shock; wave arrivals, Boysen et al. (2019).
<code>labor_utilization</code>	Classical queueing load indicator ρ .
<code>n_pickers_busy</code>	Absolute capacity remaining this epoch.

Feature	Rationale
<code>mean_pickup_time_recent</code>	Realised service-rate estimate; drifts with order mix.
<code>n_orders_late_so_far</code>	Realised failures to date; regime indicator.
<code>n_orders_at_risk_30min</code>	Count with negative slack inside 30 min — the actionable set.
<code>mean_slack_minutes</code>	Mean $d - t - p$; the ATC/MS/COVERT decision variable.
<code>std_slack_minutes</code>	Slack dispersion; separates uniform from bimodal pressure.
<code>mean_processing_time_remaining</code>	Expected work content of the queue.
<code>pct_high_priority</code>	Share of the economically weighted tail.
<code>pct_expiring_30min</code>	Product-clock analogue of <code>pct_critical</code> .
<code>mean_expiry_slack</code>	Mean $x - t - p$ over perishables; FEFO's decision variable.
<code>n_spoiled_so_far</code>	Finished spoilage to date: orders with <code>finish_time</code> $\leq t$ that are spoiled. Queued expired perishables are invisible until picked (Section 3.3 keeps them in the queue).
<code>arrival_rate_recent_60min</code>	Non-stationarity detector; empirical λ -hat.
<code>queue_length_lag_1</code>	Congestion trend; standard lag structure.
<code>queue_length_lag_2</code>	Congestion trend.
<code>queue_length_lag_3</code>	Congestion trend.
<code>failure_rate_lag_1</code>	Failure trend.
<code>failure_rate_lag_2</code>	Failure trend.
<code>failure_rate_lag_3</code>	Failure trend.
<code>interval_index_in_shift</code>	Position in the finite horizon; end-of-shift effects.

10.2. Degenerate features that were dropped

Two features are omitted from the deployed map. `time_to_next_expected_carrier` was computed as $1/\lambda$ and was therefore *constant* within any one configuration: zero variance, zero information. `intervals_remaining` was an exact affine function of `interval_index_in_shift`, the two summing to $N = 32$ by construction. Together they made the feature matrix exactly singular. That is not a cosmetic defect: the regime layer (Section 4.4) fits a full-covariance Gaussian mixture on these columns, so the covariance was singular, only the ridge term `reg_covar` prevented a failure, and each additional mixture component bought likelihood by collapsing further onto the degenerate directions.

Three features record the product clock: `pct_expiring_30min`, `mean_expiry_slack` and `n_spoiled_so_far`.

The product deadline enters the objective (Section 3.3), and a selector cannot act on a constraint it cannot observe. FEFO’s decision variable, expiry slack, has to be visible to the ranker that decides when to deploy FEFO.

One feature is named for the primary metric. The failure-rate lags are `failure_rate_lag_1..3`.

10.3. Redundancy analysis

`experiments/feature_analysis.py` runs four diagnostics on the training feature matrix: near-constant columns; Pearson and Spearman correlation with pairs above $|r| = 0.95$ flagged; variance inflation factors, obtained by regressing each feature on the remaining ones; and correlation-distance clustering with one nominated representative per cluster.

No column is near-constant. Ten pairs exceed $|r| = 0.95$, of which `labor_utilization` with `n_pickers_busy` is $r = 1$ (they encode the same capacity residual) and the failure-rate lags with `n_orders_late_so_far` are $r \geq 0.979$. Fourteen features have $VIF \geq 10$: `queue_length`, `n_arrivals_last_interval`, `n_pickers_busy`, `labor_utilization`, the three queue-length lags, the three failure-rate lags, `n_orders_late_so_far`, `mean_queue_age`, `max_queue_age`, and `mean_slack_minutes`. The clustering nominates dropping `labor_utilization`, `mean_queue_age`, `queue_length_lag_1`, `queue_length_lag_2`, and the three failure-rate lags. We did not drop them from the deployed model: the `top5_features` ablation (Section 6.8) already asks whether the long tail of the map earns its keep, and it does not, at least not detectably on 50 test shifts. The VIF column is a warning that coefficient-level stories about individual features are not identified, which is why Section 6.6 reports SHAP on the fitted trees rather than linear weights.

After dropping the two degenerate features of Section A.2, the regime BIC still selects $K^* = 12$ at the edge of $\{2, 3, 4, 5, 6, 7, 8, 10, 12\}$ (mean ARI 0.970, above the 0.85 threshold). BIC values fall from +36,871 at $K = 2$ to −240,139 at $K = 12$ and do not turn. The grid chose K , not a mode in the data. Input dimension is 38.

11. Appendix B. Configuration and hyperparameters

Every value below is read from `config.yaml` in the accompanying repository, which is the source the campaign shares, except where a grid winner is named.

Fitted Q-iteration. `config.yaml` default trees are `max_depth = 4`, `n_estimators = 200`, `learning_rate = 0.05`, $\gamma = 0.99$, 20 iterations. The E9 grid winner written to `results/E9/hp_winner.json` is `max_depth = 4`, `n_estimators = 500`, `learning_rate = 0.05`, $\gamma = 0.9$. Table 5 uses the winner.

Simulator. 8-hour shift; $N = 32$ review intervals of $L = 15$ minutes; $m = 10$ pickers; queue capacity 200 orders, with overflow recorded as rejected demand rather than discarded. Arrivals are Poisson at a nominal 1.65 orders/minute. Order attributes: processing time `Triangular(2, 5, 12)` min; customer window $d_o - a_o$ `Triangular(15, 45, 90)` min; shelf life $x_o - a_o$ `Triangular(20, 60, 120)` min for the 20% of orders that are perishable, with $x_o = \infty$ otherwise; priority classes {low, medium, high} drawn at (0.50, 0.35, 0.15) with economic weights $w_o \in \{1, 2, 4\}$. Section 3.4 gives the provenance of each.

Objective (Section 3.3). $W_b = 3.0$ per late shipment, $W_l = 0.2$ per minute of lateness, $W_s = 5.0$ per spoiled perishable, $W_h = 0.005$ per order still queued at each potential evaluation (including window labels at $t + \tau L$); shift-level J uses $|Q_T|$. Every per-order charge is multiplied by w_o . An unweighted objective with no W_s term would grade WSPT and ATC against a criterion they were not designed for, and would price an abandoned order at 0.005 against 3.0 for one served late.

Shift corpora. Seeds drawn from one SeedSequence (root 42) and partitioned into three disjoint contiguous blocks: 250 training, 30 calibration, 50 test. The calibration block exists so that ATC’s and COVERT’s look-ahead scales can be fitted without touching the training or test shifts.

Labelling (Section 4.3). Rollout horizon $\tau = 4$; $M = 20$ independent continuations per state-rule cell under common random numbers, with the per-cell standard error recorded alongside every label; behaviour policy random — not round robin, because the interval index is itself an observed feature, which would make round robin a deterministic function of the state (Section 6.10). Tempered softmax with a **per-row** temperature $T(s) = \beta \sigma(s)$, where $\sigma(s)$ is the standard deviation of that state’s own cost vector and β is searched so that the median training-label entropy falls within $[0.216, 0.505] \times \log |\mathcal{H}|$. Campaign fit: $\beta = 0.470$, median entropy 0.638 nats, band $[0.387, 0.905]$. Expiry-rule mask threshold 0.05 on the perishable fraction (a no-op on the screened pool, which contains no expiry-only rule). Test-corpus ambiguity filter at $\theta = 2.2/|\mathcal{H}|$, never applied to the training corpus. The hard-label ablation of Section 6.8 replaces the tempered softmax with the one-hot arg-max of the same cost vector and is otherwise identical.

Rule pool (Section 3.6). Nine candidates before screening: FIFO, EDD, EEDD, FEFO, WSPT, ATC, MS, MDD, COVERT. Stage 1 retained [EEDD, COVERT, MS, ATC, MDD, EDD] and fitted ATC $k^* = 3.0$ (portfolio; standalone 1.5) and COVERT $k^* = 4.0$.

Regime layer (Section 4.4). Gaussian mixture, full covariance, with K selected by BIC over $K \in \{2, 3, 4, 5, 6, 7, 8, 10, 12\}$ and 5 EM restarts per K ; stability checked by the mean adjusted Rand index over 10 refits against a 0.85 threshold. $K^* = 12$ at the grid edge, mean ARI 0.970.

Ranker. Gradient-boosted trees (Chen and Guestrin 2016), `multi:softprob` objective, sample-weighted by inverse label entropy. Hyperparameters selected from an 18-configuration grid (`max_depth` $\in \{4, 6, 8\} \times$ `n_estimators` $\in \{200, 500, 1000\} \times$ `learning_rate` $\in \{0.03, 0.1\}$) by 5-fold cross-validation grouped on `shift_id`; isotonic calibration on a 20% held-out shift split. Selected: `max_depth` = 6, `n_estimators` = 200, `learning_rate` = 0.03. Test ECE after isotonic: 0.0213 (pre 0.1700).

Switching controller (Section 4.6). Minimum dwell $T_{\min} = 2$ intervals; entropy gate at half the maximum entropy.

12. Appendix C. Fitting the input distributions to a real order trace

Source. Olist Brazilian e-commerce public dataset (Olist 2018), approximately 100k orders. Inter-arrival times are computed as within-day differences of order timestamps, which removes the multi-day growth trend that otherwise dominates the series. Because the trace is measured in days and the simulator in minutes, all comparisons are on

mean-normalised samples: the trace fixes distribution *shape*, and the operating rate is set separately.

Candidate families for inter-arrival times: exponential, lognormal, gamma, Weibull. Selected: **lognormal** ($n = 98,241$, AIC 161,984). Exponential is worse by $\Delta\text{AIC} = 34,502$. Shape tests against the simulator's Poisson default: KS $D = 0.153$, Wasserstein-1 0.443; simulated CV 1.00 versus real 2.68. Poisson is therefore an operating point, not a fitted arrival process. Section 6.7's Olist-bootstrap replay is the robustness check that uses the empirical burstiness.

Customer windows: the AIC-best family on the Olist purchase-to-estimated-delivery sample is **lognormal**. The campaign does not deploy that MLE. It uses triangular ((15, 45, 90)) as a warehouse-scale operating-point envelope. Closest shape match of the three inputs against that envelope (KS $D = 0.039$, subsampled $p = 0.022$, $W_1 = 0.035$).

Processing time: not fitted. The trace field is purchase-to-approval, not pick time. The triangular ((2, 5, 12)) is a literature three-point standard (Tompkins et al. 2010). Shape test against that proxy is correspondingly poor (KS $D = 0.686$).

Perishable fraction: design parameter 0.20 against Olist food/drink share 0.0099. Shelf life has no public analogue.

Data availability

The simulator, training pipeline, configuration and result artifacts that produced the live tables in this paper are in the accompanying repository. Table 1 is the exception: it used the eight-rule SCREENING_POOL (FIFO omitted) on 30 calibration shifts, stored at `results/S1_perishability/`. The live `resolve_pool` in `config.yaml` is the six-rule deployed pool, so reproducing Table 1 from the current config will not recover it. The data and code underlying the work are deposited as the journal's data-availability policy requires (Option C). A persistent archive with a DOI will be linked at acceptance; until then the versioned source is <https://github.com/Vittal-Mukunda/Disruption-Aware-Heuristic-Scheduling>.

CRediT authorship contribution statement

Vittal Mukunda: Conceptualization, Methodology, Software, Formal analysis, Writing — original draft. **Atharva Somani:** Software, Investigation, Validation, Data curation. **Pranjal Malaiya:** Methodology, Writing — review and editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Acknowledgements

We thank the editors and the anonymous referees for comments on an earlier version of this work.

- Baker, Kenneth R., and J. Will M. Bertrand. 1982. “A Dynamic Priority Rule for Scheduling Against Due-Dates.” *Journal of Operations Management* 3 (1): 37–42. [https://doi.org/10.1016/0272-6963\(82\)90020-1](https://doi.org/10.1016/0272-6963(82)90020-1).
- Bertsekas, Dimitri P. 2020. *Rollout, Policy Iteration, and Distributed Reinforcement Learning*. Athena Scientific.
- Boysen, Nils, and René de Koster. 2025. “50 Years of Warehousing Research—an Operations Research Perspective.” *European Journal of Operational Research* 320 (3): 449–64. <https://doi.org/10.1016/j.ejor.2024.03.026>.
- Boysen, Nils, René de Koster, and Felix Weidinger. 2019. “Warehousing in the e-Commerce Era: A Survey.” *European Journal of Operational Research* 277 (2): 396–411. <https://doi.org/10.1016/j.ejor.2018.08.023>.
- Branke, Jürgen, Su Nguyen, Christoph W. Pickardt, and Mengjie Zhang. 2016. “Automated Design of Production Scheduling Heuristics: A Review.” *IEEE Transactions on Evolutionary Computation* 20 (1): 110–24. <https://doi.org/10.1109/TEVC.2015.2429314>.
- Carroll, Donald C. 1965. “Heuristic Sequencing of Single and Multiple Component Jobs.” PhD thesis, Massachusetts Institute of Technology.
- Chen, Tianqi, and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–94. <https://doi.org/10.1145/2939672.2939785>.
- Chen, Wei, Zequn Zhang, Dunbing Tang, et al. 2024. “Probing an LSTM-PPO-Based Reinforcement Learning Algorithm to Solve Dynamic Job Shop Scheduling Problem.” *Computers & Industrial Engineering* 197: 110633. <https://doi.org/10.1016/j.cie.2024.110633>.
- Cheng, Bayi, Lingjun Wang, Qi Tan, and Mi Zhou. 2024. “A Deep Reinforcement Learning Hyper-Heuristic to Solve Order Batching Problem with Mobile Robots.” *Applied Intelligence* 54: 6865–87. <https://doi.org/10.1007/s10489-024-05532-9>.
- Conway, Richard W., William L. Maxwell, and Louis W. Miller. 1967. *Theory of Scheduling*. Addison-Wesley.

- Dimitrakakis, Christos, and Michail G. Lagoudakis. 2008. "Rollout Sampling Approximate Policy Iteration." *Machine Learning* 72 (3): 157–71. <https://doi.org/10.1007/s10994-008-5069-3>.
- Dokeroglu, Tansel, Tayfun Kucukyilmaz, and El-Ghazali Talbi. 2024. "Hyper-Heuristics: A Survey and Taxonomy." *Computers & Industrial Engineering* 187: 109815. <https://doi.org/10.1016/j.cie.2023.109815>.
- Drake, John H., Ahmed Kheiri, Ender Özcan, and Edmund K. Burke. 2020. "Recent Advances in Selection Hyper-Heuristics." *European Journal of Operational Research* 285 (2): 405–28. <https://doi.org/10.1016/j.ejor.2019.07.073>.
- Durašević, Marko, and Domagoj Jakobović. 2022. "Selection of Dispatching Rules Evolved by Genetic Programming in Dynamic Unrelated Machines Scheduling Based on Problem Characteristics." *Journal of Computational Science* 61: 101649. <https://doi.org/10.1016/j.jocs.2022.101649>.
- Ernst, Damien, Pierre Geurts, and Louis Wehenkel. 2005. "Tree-Based Batch Mode Reinforcement Learning." *Journal of Machine Learning Research* 6: 503–56.
- Farahmand, Amir-massoud, Doina Precup, André M. S. Barreto, and Mohammad Ghavamzadeh. 2015. "Classification-Based Approximate Policy Iteration." *IEEE Transactions on Automatic Control* 60 (11): 2989–93. <https://doi.org/10.1109/TAC.2015.2418411>.
- Fern, Alan, Sungwook Yoon, and Robert Givan. 2006. "Approximate Policy Iteration with a Policy Language Bias: Solving Relational Markov Decision Processes." *Journal of Artificial Intelligence Research* 25: 75–118. <https://doi.org/10.1613/jair.1700>.
- Geng, Xin. 2016. "Label Distribution Learning." *IEEE Transactions on Knowledge and Data Engineering* 28 (7): 1734–48. <https://doi.org/10.1109/TKDE.2016.2545658>.
- Gils, Teun van, Katrien Ramaekers, An Caris, and René B. M. de Koster. 2018. "Designing Efficient Order Picking Systems by Combining Planning Problems: State-of-the-Art Classification and Review." *European Journal of Operational Research* 267 (1): 1–15. <https://doi.org/10.1016/j.ejor.2017.09.002>.
- Goodson, Justin C., Barrett W. Thomas, and Jeffrey W. Ohlmann. 2016. "Restocking-Based Rollout Policies for the Vehicle Routing Problem with Stochastic Demand and Duration Limits." *Transportation Science* 50 (2): 591–607. <https://doi.org/10.1287/trsc.2015.0591>.
- Goodson, Justin C., Barrett W. Thomas, and Jeffrey W. Ohlmann. 2017. "A Rollout Algorithm Framework for Heuristic Solutions to Finite-Horizon Stochastic Dynamic Programs." *European Journal of Operational Research* 258 (1): 216–29. <https://doi.org/10.1016/j.ejor.2016.09.040>.
- Han, Young-Suk, and Jae-Yoon Jung. 2025. "Imitation Learning of Job Dispatching Data Combining Classification and Ranking." *Computers & Industrial Engineering* 207: 111305. <https://doi.org/10.1016/j.cie.2025>.

111305.

- Jackson, James R. 1955. *Scheduling a Production Line to Minimize Maximum Tardiness*. Research Report 43. Management Sciences Research Project, University of California, Los Angeles.
- Klapp, Mathias A., Alan L. Erera, and Alejandro Toriello. 2018a. “The Dynamic Dispatch Waves Problem for Same-Day Delivery.” *European Journal of Operational Research* 271 (2): 519–34. <https://doi.org/10.1016/j.ejor.2018.05.032>.
- Klapp, Mathias A., Alan L. Erera, and Alejandro Toriello. 2018b. “The One-Dimensional Dynamic Dispatch Waves Problem.” *Transportation Science* 52 (2): 402–15. <https://doi.org/10.1287/trsc.2016.0682>.
- Koster, René de, Tho Le-Duc, and Kees Jan Roodbergen. 2007. “Design and Control of Warehouse Order Picking: A Literature Review.” *European Journal of Operational Research* 182 (2): 481–501.
- Lagoudakis, Michail G., and Ronald Parr. 2003. “Reinforcement Learning as Classification: Leveraging Modern Classifiers.” *Proceedings of the 20th International Conference on Machine Learning (ICML)*, 424–31.
- Law, Averill M., and W. David Kelton. 2000. *Simulation Modeling and Analysis*. 3rd ed. McGraw-Hill.
- Li, Lihong, Wei Chu, John Langford, and Robert E. Schapire. 2010. “A Contextual-Bandit Approach to Personalized News Article Recommendation.” *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 661–70. <https://doi.org/10.1145/1772690.1772758>.
- Liu, Anbang, Shaochong Lin, Jingchuan Chen, Peng Wu, and Zuojun Max Shen. 2025. *Machine Learning for Scheduling: A Paradigm Shift from Solver-Centric to Data-Centric Approaches*. <https://arxiv.org/abs/2512.22642>.
- Lundberg, Scott M., and Su-In Lee. 2017. “A Unified Approach to Interpreting Model Predictions.” *Advances in Neural Information Processing Systems* 30.
- Mahmoudinazlou, Sasan, Abhay Sobhanan, Hadi Charkhgard, Ali Eshragh, and George Dunn. 2025. “Deep Reinforcement Learning for Dynamic Order Picking in Warehouse Operations.” *Computers & Operations Research* 182: 107112. <https://doi.org/10.1016/j.cor.2025.107112>.
- Mouelhi-Chibani, Wiem, and Henri Pierreval. 2010. “Training a Neural Network to Select Dispatching Rules in Real Time.” *Computers & Industrial Engineering* 58 (2): 249–56. <https://doi.org/10.1016/j.cie.2009.03.008>.
- Nguyen, Su, Yi Mei, and Mengjie Zhang. 2017. “Genetic Programming for Production Scheduling: A Survey with a Unified Framework.” *Complex & Intelligent Systems* 3 (1): 41–66. <https://doi.org/10.1007/s40747-017-0036-x>.
- Olist. 2018. *Brazilian e-Commerce Public Dataset by Olist*. Kaggle.

- Pinedo, Michael L. 2016. *Scheduling: Theory, Algorithms, and Systems*. 5th ed. Springer. <https://doi.org/10.1007/978-3-319-26580-3>.
- Powell, Warren B. 2019. "A Unified Framework for Stochastic Optimization." *European Journal of Operational Research* 275 (3): 795–821. <https://doi.org/10.1016/j.ejor.2018.07.014>.
- Powell, Warren B. 2022. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. Wiley.
- Remmerden, Jesse van, Zaharah Bukhsh, and Yingqian Zhang. 2025. "Offline Reinforcement Learning for Learning to Dispatch for Job Shop Scheduling." *Machine Learning* 114: 191. <https://doi.org/10.1007/s10994-025-06826-w>.
- Roodbergen, Kees Jan, and Iris F. A. Vis. 2006. "A Model for Warehouse Layout." *IIE Transactions* 38 (10): 799–811.
- Sargent, Robert G. 2013. "Verification and Validation of Simulation Models." *Proceedings of the Winter Simulation Conference*.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. *Proximal Policy Optimization Algorithms*. <https://arxiv.org/abs/1707.06347>.
- Shiue, Yeou-Ren, Ken-Chuan Lee, and Chao-Ton Su. 2020. "A Reinforcement Learning Approach to Dynamic Scheduling in a Product-Mix Flexibility Environment." *IEEE Access* 8: 106542–53. <https://doi.org/10.1109/ACCESS.2020.3000781>.
- Smith, Wayne E. 1956. "Various Optimizers for Single-Stage Production." *Naval Research Logistics Quarterly* 3 (1-2): 59–66. <https://doi.org/10.1002/nav.3800030106>.
- Stöckermann, Patrick, Henning Südfeld, Alessandro Immordino, et al. 2025. "Scalability of Reinforcement Learning Methods for Dispatching in Semiconductor Frontend Fabs: A Comparison of Open-Source Models with Real Industry Datasets." *The International Journal of Advanced Manufacturing Technology*, ahead of print. <https://doi.org/10.1007/s00170-025-16117-2>.
- Tassel, Pierre, Martin Gebser, and Konstantin Schekotihin. 2023. "An End-to-End Reinforcement Learning Approach for Job-Shop Scheduling Problems Based on Constraint Programming." *Proceedings of the International Conference on Automated Planning and Scheduling* 33 (1): 614–22. <https://doi.org/10.1609/icaps.v33i1.27243>.
- Tompkins, James A., John A. White, Yavuz A. Bozer, and J. M. A. Tanchoco. 2010. *Facilities Planning*. 4th ed. John Wiley & Sons.
- Ulmer, Marlin W. 2019. "Anticipation Versus Reactive Reoptimization for Dynamic Vehicle Routing with Stochastic Requests." *Networks* 73 (3): 277–91. <https://doi.org/10.1002/net.21861>.

- Ulmer, Marlin W., Justin C. Goodson, Dirk C. Mattfeld, and Marco Hennig. 2019. "Offline–Online Approximate Dynamic Programming for Dynamic Vehicle Routing with Stochastic Requests." *Transportation Science* 53 (1): 185–202. <https://doi.org/10.1287/trsc.2017.0767>.
- Ulmer, Marlin W., Justin C. Goodson, Dirk C. Mattfeld, and Barrett W. Thomas. 2020. "On Modeling Stochastic Dynamic Vehicle Routing Problems." *EURO Journal on Transportation and Logistics* 9 (2): 100008. <https://doi.org/10.1016/j.ejtl.2020.100008>.
- Ulmer, Marlin W., Dirk C. Mattfeld, and Felix Köster. 2018. "Budgeting Time for Dynamic Vehicle Routing with Stochastic Customer Requests." *Transportation Science* 52 (1): 20–37. <https://doi.org/10.1287/trsc.2016.0719>.
- Vepsäläinen, Ari P. J., and Thomas E. Morton. 1987. "Priority Rules for Job Shops with Weighted Tardiness Costs." *Management Science* 33 (8): 1035–47. <https://doi.org/10.1287/mnsc.33.8.1035>.
- Winkelhaus, Sven, and Eric H. Grosse. 2020. "Logistics 4.0: A Systematic Review Towards a New Logistics System." *International Journal of Production Research* 58 (1): 18–43. <https://doi.org/10.1080/00207543.2019.1612964>.
- Wu, S. David, and Richard A. Wysk. 1988. "Multi-Pass Expert Control System — a Control/Scheduling Structure for Flexible Manufacturing Cells." *Journal of Manufacturing Systems* 7 (2): 107–20. [https://doi.org/10.1016/0278-6125\(88\)90018-0](https://doi.org/10.1016/0278-6125(88)90018-0).